UNSW SYDNEY

Detecting Event-Based Races in Mobile Applications

Author:
Wu, Diyu

Publication Date:
2021

DOI:
https://doi.org/10.26190/unsworks/22354

License:
https://creativecommons.org/licenses/by-nc-nd/3.0/au/
Link to license to see what you are allowed to do with this resource.

Downloaded from http://hdl.handle.net/1959.4/70649 in https://unsworks.unsw.edu.au on 2026-04-20

Detecting Event-Based Races in Mobile Applications

by

Diyu Wu

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF

THE REQUIREMENTS FOR THE DEGREE OF

DOCTOR OF PHILOSOPHY

IN THE SCHOOL

OF

COMPUTER SCIENCE AND ENGINEERING

THE UNIVERSITY OF
NEW SOUTH WALES

SYDNEY · AUSTRALIA

February 16, 2021

All rights reserved. This work may not be

reproduced in whole or in part, by photocopy

or other means, without the permission of the author.

© Diyu Wu 2020

🎓 Welcome to the Research Alumni Portal, Diyu Wu!

You will be able to download the finalised version of all thesis submissions that were processed in GRIS here.

Please ensure to include the **completed declaration** (from the Declarations tab), your **completed Inclusion of Publications Statement** (from the Inclusion of Publications Statement tab) in the final version of your thesis that you submit to the Library.

Information on how to submit the final copies of your thesis to the Library is available in the completion email sent to you by the GRS.

Thesis submission for the degree of Doctor of Philosophy

Thesis Title and Abstract | **Declarations** | **Inclusion of Publications Statement** | **Corrected Thesis and Responses**

Thesis Title

Detecting Event-Based Races in Mobile Applications

Thesis Abstract

Modern mobile platforms have formed their distinct hybrid concurrency system containing both traditional multi-threading and asynchronous event dispatching to support a diverse range of applications. However, this system introduces prevalent concurrency bugs, causing misbehaviour of mobile apps. It is challenging to detect concurrency bugs in mobile apps. Static analysis is prone to a high false positive rate due to the complexity of the hybrid concurrency system, while dynamic analysis suffers from the code coverage problem and the existence of conditional statements in programs creates obstacles to dynamically reaching the target statements. This thesis presents two new techniques for detecting event-based races in Android apps, where Android is the dominant mobile platform and has been plagued by event-based races.

First, a novel static analysis tool, called Sard is introduced for detecting event-based races causing use-after-free violations in Android apps. This tool systematically models Android's concurrency mechanism and applies a novel flow- and context-sensitive static happens-before analysis to reason the interleavings relations between different events and threads. Compared to the prior state-of-the-art tool, Sard achieves better precision (by reporting less false positives and false negatives) and better efficiency.

Second, we propose Sieve, which combines both static and dynamic analysis to detect event-based races in Android apps. Sieve first applies static analysis to find all suspicious races, and then attempts to expose these races dynamically by leveraging a new selective branch instrumentation. For the conditionals potentially affecting a suspicious race, Sieve fixes the true/false outcomes of some of these conditionals based on a systematic branch analysis. By instrumenting certain branches selectively this way, Sieve can not only expose event-based races more effectively than the prior state-of-the-art techniques, but also substantially reduce the negative ramifications of instrumentation.

This thesis presents the methodologies and implementations of both proposed tools and evaluates them using real-world Android apps. The experimental results demonstrate that both Sard and Sieve have met their design goals and bring inspiration on how to perform program analysis on detecting event-based races in event-driven mobile platforms.

🎓 Welcome to the Research Alumni Portal, Diyu Wu!

You will be able to download the finalised version of all thesis submissions that were processed in GRIS here.

Please ensure to include the **completed declaration** (from the Declarations tab), your **completed Inclusion of Publications Statement** (from the Inclusion of Publications Statement tab) in the final version of your thesis that you submit to the Library.

Information on how to submit the final copies of your thesis to the Library is available in the completion email sent to you by the GRS.

Thesis submission for the degree of Doctor of Philosophy

Thesis Title and Abstract | **Declarations** | **Inclusion of Publications Statement** | **Corrected Thesis and Responses**

ORIGINALITY STATEMENT

☑ I hereby declare that this submission is my own work and to the best of my knowledge it contains no materials previously published or written by another person, or substantial proportions of material which have been accepted for the award of any other degree or diploma at UNSW or any other educational institution, except where due acknowledgement is made in the thesis. Any contribution made to the research by others, with whom I have worked at UNSW or elsewhere, is explicitly acknowledged in the thesis. I also declare that the intellectual content of this thesis is the product of my own work, except to the extent that assistance from others in the project's design and conception or in style, presentation and linguistic expression is acknowledged.

COPYRIGHT STATEMENT

☑ I hereby grant the University of New South Wales or its agents a non-exclusive licence to archive and to make available (including to members of the public) my thesis or dissertation in whole or part in the University libraries in all forms of media, now or here after known. I acknowledge that I retain all intellectual property rights which subsist in my thesis or dissertation, such as copyright and patent rights, subject to applicable law. I also retain the right to use all or part of my thesis or dissertation in future works (such as articles or books).

For any substantial portions of copyright material used in this thesis, written permission for use has been obtained, or the copyright material is removed from the final public version of the thesis.

AUTHENTICITY STATEMENT

☑ I certify that the Library deposit digital copy is a direct equivalent of the final officially approved version of my thesis.

🎓 Welcome to the Research Alumni Portal, Diyu Wu!

You will be able to download the finalised version of all thesis submissions that were processed in GRIS here.

Please ensure to include the **completed declaration** (from the Declarations tab), your **completed Inclusion of Publications Statement** (from the Inclusion of Publications Statement tab) in the final version of your thesis that you submit to the Library.

Information on how to submit the final copies of your thesis to the Library is available in the completion email sent to you by the GRS.

Thesis submission for the degree of Doctor of Philosophy

Thesis Title and Abstract | **Declarations** | **Inclusion of Publications Statement** | **Corrected Thesis and Responses**

UNSW is supportive of candidates publishing their research results during their candidature as detailed in the UNSW Thesis Examination Procedure.

Publications can be used in the candidate's thesis in lieu of a Chapter provided:

- The candidate contributed **greater than 50%** of the content in the publication and are the "primary author", i.e. they were responsible primarily for the planning, execution and preparation of the work for publication.
- The candidate has obtained approval to include the publication in their thesis in lieu of a Chapter from their Supervisor and Postgraduate Coordinator.
- The publication is not subject to any obligations or contractual agreements with a third party that would constrain its inclusion in the thesis.

☑ The candidate has declared that **their thesis has publications - either published or submitted for publication - incorporated into it in lieu of a Chapter/s. Details of these publications are provided below.**.

Publication Details #1

Full Title:	Precise Static Happens-Before Analysis for Detecting UAF Order Violations in Android
Authors:	Diyu Wu, Jie Liu, Yulei Sui, Shiping Chen and Jingling Xue
Journal or Book Name:	12th IEEE International Conference on Software Testing, Verification and Validation (ICST'19)
Volume/Page Numbers:	276-287
Date Accepted/Published:	22/04/2019
Status:	published

The Candidate's Contribution to the Work:	The candidate Diyu Wu contributed greater than 70% of the content in the publication and is the primary author of this publication.
Location of the work in the thesis and/or how the work is incorporated in the thesis:	Chapter 3

Publication Details #2

Full Title:	Exposing Android Event-Based Races by Selective Branch Instrumentation
Authors:	Diyu Wu, Dongjie He, Shiping Chen and Jingling Xue
Journal or Book Name:	31st IEEE International Symposium on Software Reliability Engineering (ISSRE'20)
Volume/Page Numbers:	
Date Accepted/Published:	
Status:	accepted
The Candidate's Contribution to the Work:	The candidate Diyu Wu contributed greater than 70% of the content in the publication and is the primary author of this publication.
Location of the work in the thesis and/or how the work is incorporated in the thesis:	Chapter 4

Candidate's Declaration

I confirm that where I have used a publication in lieu of a chapter, the listed publication(s) above meet(s) the requirements to be included in the thesis. I also declare that I have complied with the Thesis Examination Procedure.

Abstract

Modern mobile platforms have formed their distinct hybrid concurrency system containing both traditional multi-threading and asynchronous event dispatching to support a diverse range of applications. However, this system introduces prevalent concurrency bugs, causing misbehaviour of mobile apps. It is challenging to detect concurrency bugs in mobile apps. Static analysis is prone to a high false positive rate due to the complexity of the hybrid concurrency system, while dynamic analysis suffers from the code coverage problem and the existence of conditional statements in programs creates obstacles to dynamically reaching the target statements. This thesis presents two new techniques for detecting event-based races in Android apps, where Android is the dominant mobile platform and has been plagued by event-based races.

First, a novel static analysis tool, called Sard is introduced for detecting event-based races causing use-after-free violations in Android apps. This tool systematically models Android's concurrency mechanism and applies a novel flow- and context-sensitive static happens-before analysis to reason the interleaving relations between different events and threads. Compared to the prior state-of-the-art tool, Sard achieves better precision (by reporting less false positives and false negatives) and better efficiency.

Second, we propose Sieve, which combines both static and dynamic analysis to detect event-based races in Android apps. Sieve first applies static analysis to find all suspicious races, and then attempts to expose these races dynamically by leveraging a new selective branch instrumentation. For the conditionals potentially affecting a suspicious race, Sieve fixes the true/false outcomes of some of these conditionals based on a systematic branch analysis. By instrumenting certain branches selectively this way, Sieve can not only expose event-based races more effectively than the prior state-of-the-art techniques, but also substantially reduce the negative ramifications of instrumentation.

This thesis presents the methodologies and implementations of both proposed tools and evaluates them using real-world Android apps. The experimental results demonstrate that both Sard and Sieve have met their design goals and bring inspiration on how to perform program analysis on detecting event-based races in event-driven mobile platforms.

Publications

- **Diyu Wu**, Dongjie He, Shiping Chen and Jingling Xue. Exposing Android Event-Based Races by Selective Branch Instrumentation. Published in *31st IEEE International Symposium on Software Reliability Engineering (ISSRE'20)*.

- **Diyu Wu**, Jie Liu, Yulei Sui, Shiping Chen and Jingling Xue. Precise Static Happens-Before Analysis for Detecting UAF Order Violations in Android. Published in *12th IEEE International Conference on Software Testing, Verification and Validation (ICST'19)*.

  Jie Liu, Dongjie He, **Diyu Wu** and Jingling Xue. Correlating UI Contexts with Sensitive API Calls: Dynamic Semantic Extraction and Analysis Published in *31st IEEE International Symposium on Software Reliability Engineering (ISSRE'20)*.

- Jie Liu, **Diyu Wu** and Jingling Xue. TDroid: Exposing App Switching Attacks in Android with Control Flow Specialization. Published in *33rd IEEE/ACM International Conference on Automated Software Engineering (ASE'18)*.

Acknowledgments

Undertaking a PhD has been a life-changing experience for me and would not have been possible without the support and help I received from many people.

First, I would like to express my sincere thanks to my supervisor, Professor Jingling Xue, who was always patient and helpful throughout my study. I appreciate the time he spent with me discussing my research ideas and polishing my writing. His insight always clarified problems and ensured I stayed on the right track during my study, and his rigorous, persevering altitude and enthusiasm towards research set a great model for my future career.

I would also like to thank the other people who provided supervision. Dr. Shiping Chen offered a great opportunity to collaborate with Data61 CSIRO and provided encouragement for my work on a very challenging topic. Dr. Yue Li revealed the world of research to me and gave me precious advice that benefited both my research and my life. Dr. Yulei Sui dedicated his time to my very first paper and helped me through hard times with his knowledge of program analysis and excellent writing skills.

My special gratitude goes to Dr. Jie Liu, a colleague and a best friend. Not only did he contribute valuable advice for my research, he also provided endless encouragement and shared his experience with me during a tough time.

I am also thankful to all the other members of the CORG group, past and present—Xinwei Xie, Peng Di, Ding Ye, Xiaokang Fan, Tian Tan, Hua Yan, Feng Zhang, Yifei Zhang, Jieyuan Zhang, Jingbo Lu, Xuezheng Xu, Changwei Zou, Dongjie He and Xudong Wang. It was pleasant to work in such a friendly environment provided by you.

Last, but most importantly, I express my gratefulness to my parents and my girlfriend. I am aware that no matter what happens, their accompanying and unconditional support will always be there, letting me rest and recover. It is my blessing to have them in my life and I dedicate this thesis to them.

Contents

Abstract **vii**

Publications **ix**

Acknowledgments **xi**

List of Figures **xviii**

List of Tables **xix**

1 Introduction **1**

1.1 Motivation 1

1.2 Challenges 2

1.3 Contributions 4

1.4 Thesis Structure 7

2 Background **9**

2.1 Android Overview 9

2.1.1 Architecture 9

2.1.2 App Components 12

2.2 Android Concurrency 13

2.2.1 Processes and Threads 13

2.2.2 Event Dispatching System 14
2.2.3 Non-Determinism 16
2.2.4 Happens-Before Relations 17
2.3 Event-Based Races 17

3 Precise Happens-Before Analysis 21
3.1 Overview 21
3.2 Related Work 25
3.3 A Motivating Example 26
3.3.1 Existing Techniques 28
3.3.2 Sard 29
3.4 Approach 31
3.4.1 Abstract Event Modeling 31
3.4.2 Static Happens-Before Analysis 36
3.4.3 Static Order Violation Detection 38
3.5 Evaluation 40
3.5.1 Implementation 40
3.5.2 Methodology 40
3.5.3 RQ1. Effectiveness and Efficiency of Sard 41
3.5.4 RQ2. Sard vs. nAdroid: Effectiveness and Efficiency . . . 44
3.5.5 RQ3. Sard vs. nAdroid: False Negatives 47
3.5.6 Limitations 49
3.6 Chapter Summary 49

4 Selective Branch Instrumentation 51
4.1 Overview 51
4.2 Related Work 54

4.3 A Motivating Example 55
4.3.1 Existing Techniques 56
4.3.2 Sieve 57
4.4 Approach 59
4.4.1 Overview 59
4.4.2 Collecting Conditionals 61
4.4.3 Branch Analysis 62
4.4.4 Selective Branch Instrumentation 66
4.4.5 Dynamic Execution 69
4.5 Evaluation 70
4.5.1 Implementation 71
4.5.2 Methodology 71
4.5.3 Results and Analysis 72
4.5.4 Limitations 81
4.6 Chapter Summary 82

5 Conclusions and Future Work **83**

Bibliography **85**

List of Figures

2.1 Android platform architecture 10

2.2 Event dispatching in Android. 14

2.3 An event-based race example in `ConnectBot` 18

3.1 A motivating example of Sard 27

3.2 Workflow of Sard. 30

3.3 Modeling abstract events based on API calls. 32

3.4 Rules for modeling abstract events. 33

3.5 Rules for building happens-before relations. 35

3.6 Commonly occurring infeasible paths. 39

3.7 Pie charts for visualizing the data in Table 3.1. 44

3.8 A true order violation in `MyTracks_2`. 47

4.1 A motivating example of Sieve. 56

4.2 Workflow of Sieve. 60

4.3 Rules for collecting conditionals. 61

4.4 Rules for definition tracing. 64

4.5 Rules for value range analysis. 65

4.6 Rules for selective branch instrumentation. 67

4.7 An inter-thread event-based race. 69

4.8 Different causes for unreached races. 75

4.9 A true race in `VitoshaDM` 77

4.10 Comparing SIEVE and SIEVE-ZB in terms of their reach rates. . . . 79

4.11 A false positivie in `Ringdroid` 80

List of Tables

3.1 The effectiveness and efficiency of SARD in analyzing 27 real-world Android apps. 42

3.2 Comparing SARD and NADROID in effectiveness and efficiency. . . . 45

3.3 Comparing SARD and NADROID in false negatives 48

4.1 Comparing SIEVE with SIEVE-ZB and SIEVE-FB 73

Chapter 1

Introduction

1.1 Motivation

Mobile phones, which are ubiquitous in the modern world, have changed the way billions of people live. As of January 2019, it is reported that there are over 5 billion unique mobile users around the world, and this number is still increasing [5]. Mobile users rely on applications available on mobile platforms for various tasks (e.g., obtaining information, communicating with others and for entertainment). To support a variety of sensors and user input modalities, which can allow the apps rich functionalities, asynchronous event dispatching systems are naturally applied to mobile platforms. Together with the traditional multi-thread mechanism, this forms the basis of mobile platforms' distinct hybrid concurrency systems. Although capable of supporting diverse apps, such hybrid systems introduce more complex concurrency issues.

This thesis focuses on Android, which is the dominant operating system for mobile devices [10]. The official app store of Android, Google Play [7], now features around 3 million apps [11], most of which are plagued by concurrency bugs. Ac-

cording to the report [83], concurrency has always been one of the top 3 bug topics since 2008. A study of 18000 fixed bugs in Android [82] demonstrated that 66% of the high-severity bugs are concurrency related. Because of such prevalence, there is a strong impetus for developing program analysis tools competent for detecting concurrency bugs in mobile platforms with a hybrid concurrency system, such as Android, to avoid crashes, misbehaviour or even security vulnerabilities.

1.2 Challenges

Although most Android apps are implemented in Java, detecting concurrency bugs in Android apps differs greatly from traditional Java programs. Unlike a traditional Java program, there is no dedicated `main()` method as the program entry of an Android app. Instead, the execution of an app is driven by asynchronous events, which could be generated externally (by sensor inputs or system notifications) or internally (via calling certain APIs). For each generated event, the Android operating system will dispatch it later to execute a responsible method (referred to as an event method). Even if these events are processed sequentially in each thread, the execution order of most event methods is non-deterministic, because (1) the sensor inputs or system notifications are unpredictable, and (2) the internally generated events could be created by other events from different background threads. Due to the lack of synchronization, most of the event methods are logically concurrent to each other even if they are executed in the same thread. Therefore, there are not only traditional multi-thread bugs in Android, but also event-based concurrency bugs that may include only one thread.

Based on the research [51], event-based concurrency bugs in Android are four to seven times more frequent than traditional multi-thread concurrency bugs. There-

fore, an effective detection tool for Android must be able to detect event-based concurrency bugs. Unfortunately, most current Java-based concurrency bug detection tools [18, 21, 22, 24, 25, 47, 56, 59, 60, 63, 64, 72, 73, 76, 78–80] do not model the event dispatching system in Android. Thus, directly applying those tools on Android apps does not work ideally because they are unable to detect event-based concurrency bugs. Recently, researchers have proposed dedicated tools [16, 23, 26, 31–33, 51, 62, 70] for Android concurrency bug detection. Although able to find real event-based concurrency bugs, these tools still have substantial problems in real-world usage.

Dynamic tools [16, 31, 51] were the first to be proposed. They proceed in a similar fashion by exploring the apps on an instrumented device in order to collect runtime traces, which contain concurrency related operations (event begin/end/-posting, thread fork/join/wait/notify and memory read/write). Based on the collected traces, these tools apply their own algorithms and *happens-before* rules to eventually detect concurrency bugs. Dynamic tools are able to detect true bugs with relatively less false positives but suffer from code coverage problem because of their dynamic nature, which degrades their soundness. In addition, extra efforts are required for obtaining high-quality exploration strategies to ensure the effectiveness of these tools.

In pursuit of better soundness, researchers have paid their attention to static tools [26, 32, 62]. These tools statically locate the statements accessing same memory locations that can be executed from different event methods, whose execution order is non-deterministic. They then prune their results with happens-before relations built according to their model of Android's concurrency system. However, the complexity of the hybrid system makes it is difficult to precisely reason the happens-before relations, resulting in a lot of false positives.

There are also tools that combine static and dynamic analysis [23, 33, 70]. These tools first statically detect suspicious bugs or take the results of other tools as inputs, then expose as many true concurrency bugs as possible by performing dynamic execution. In this way, they have advantages of both worlds: better soundness than dynamic analysis and fewer false positives than static analysis. The challenge of this kind of approach lies in how to better reach the target statements during the dynamic execution with the existence of numerous conditional statements. In real-world apps, the statements that may cause concurrency bugs are usually guarded by conditionals so that their outcomes are affected by some other events. Due to the exponential number of events that must be tried for an app, it is not easy to exercise all the related conditionals and reach the target statements.

1.3 Contributions

Noting the aforementioned challenges, we propose two approaches for detecting event-based races in Android. First, we introduce a novel static happens-before analysis aiding static analysis to build happens-before relations between events under the complicated environment with better precision than previous approaches. We also propose a new selective branch instrumentation technique that helps the hybrid static-dynamic analysis overcome the challenge of conditional statements preventing the dynamic execution from reaching target statements. Based on our approaches, we develop one static tool Sard and one hybrid static-dynamic tool Sieve. The original contributions are as follows:

- **Precise static happens-before analysis.** This study proposes a precise static happens-before analysis for detecting event-based races causing use-after-free (UAF) violations (also referred to as UAF order violations). Due

to non-deterministic execution order between two event methods, a pointer may be dereferenced (used) after it no longer points to any valid object. In this case, a null pointer exception will be thrown, causing an unexpected termination for the app.

Because of the complexity of the hybrid concurrency system in Android, existing static analysis tools [26, 32, 62] have difficulties in precisely modelling asynchronous events under infinite event interleavings, yet are not able to precisely identify happens-before relations between different events, whose event methods may run in different background threads. To address these limitations, we propose a new flow- and context-sensitive static happens-before analysis. Our context-sensitive analysis contributes to distinguishing the calling contexts of each event to detect where an event is created and the running thread of its event method. The flow-sensitive analysis precisely reasons about the control-flow execution order inside an atomic method for determining event-creating orders. With our new happens-before analysis, not only can we identify more happens-before relations to reduce false positives, but we are also able to remove spurious relations from previous tools, thereby discovering more true event-based races.

Based on our new happens-before analysis, we introduce Sard, a novel static tool designed to detect UAF order violations in Android apps. Sard first systematically models the abstract events in an app and then performs flow- and context-sensitive analysis to infer the happens-before relations for all the abstract events. Sard then detects order violations between two events based on their happens-before relations and the field usages in the two event methods. In addition, Sard applies a lightweight feasible path analysis to further prune out false alarms by identifying spurious UAF pairs. Sard is

built upon FlowDroid [14], a static taint analysis tool for Android. We evaluated Sard against a state-of-the-art static tool, nAdroid [26] with the same set of 27 real-world Android apps. Our results reveal that Sard significantly outperforms nAdroid in terms of both precision (by reporting three times fewer false alarms and three more true alarms given the same set of apps) and efficiency (by running two orders of magnitude faster).

- **Selective branch instrumentation.** To further reduce the false positives generated by static tools and acquire the advantages of both static and dynamic analysis, one may apply first static analysis to find all the suspicious event-based races on some racy statements (where the event-based races may happen) in an app and then expose as many true races as possible by performing dynamic analysis. However, the complicated conditional statements in programs may prevent dynamic execution from reaching the racy statements. For the purposes of satisfying those conditionals, we present a novel selective branch instrumentation technique. We systematically analyze each branch to decide whether a conditional that can affect a suspicious race should be instrumented or not (i.e., specialized to be always taken or not). With the instrumentation on every selected branch, we are able to steer the program execution towards the racy statements under consideration.

  The key novelty behind our approach is that we apply a systematic branch analysis, including value range analysis and safeness analysis, to instrument selectively some conditionals guarding a suspicious race. For every such conditional, analyzing the possible values of the variables used can reveal whether the conditional may be satisfied or not under a certain context, hence allowing us to avoid introducing non-existent races. Analyzing the safeness of instrumentation on every conditional reduces unexpected crashes during dynamic

execution. Therefore, not only can we effectively expose event-based races, we can also substantially reduce the negative ramifications of instrumentation.

Leveraging selective branch instrumentation, we introduce Sieve, a hybrid static-dynamic tool for detecting event-based races. For a given app, Sieve first leverages Sard to find suspicious races statically. Based on those races, Sieve performs a branch analysis on the conditionals guarding the racy statements and selectively instruments them with fixed value. Sieve then executes the instrumented app to expose the races dynamically. Sieve is also built upon FlowDroid [14]. In addition, it leverages the help of Gator [77], Android Debugging Bridge (adb) [1] and Logcat [9]. As the previous state-of-the-art APEChecker [23] is not open-sourced, we evaluated Sieve against a baseline tool (implemented by ourselves) that imitates how APEChecker works. Our set of benchmarks consists of 25 real-world Android apps containing 190 suspicious event-based races, of which 18 are true races. Among all the suspicious races, Sieve successfully reached 90 races (47.3%) during the dynamic execution and exposed 16 true races without any false alarm, while the baseline tool reached only 56 (29.4%) races and found only 14 true races.

1.4 Thesis Structure

The remainder of this thesis is organized as follows. In Chapter 2, we introduce the background information of Android framework together with its hybrid concurrency system and an event-based race example in a real-world app. In Chapter 3, we introduce a new static tool for detecting UAF order violations. Chapter 4 describes a new hybrid static-dynamic tool for exposing event-based races. Finally, Chapter 5

summarizes the thesis and discusses potential future improvements to the proposed tools.

Chapter 2

Background

This chapter introduces relevant background information that is helpful for comprehending this thesis. First, Section 2.1 briefly introduces the basics of Android, including Android platform architecture and Android app components. Section 2.2 describes the concurrency system in Android. Section 2.3 discusses the event-based races in Android apps with an example.

2.1 Android Overview

2.1.1 Architecture

Android is an open-source operating system developed based on a modified version of the Linux kernel. Figure 2.1 demonstrates the architecture and the major components of the Android framework. We describe the functions of each layer from the bottom.

Linux Kernel. As the foundation of the Android framework, the Linux kernel offers the basic services of an operating system such as power management, memory management, file system and process and thread management, etc. By

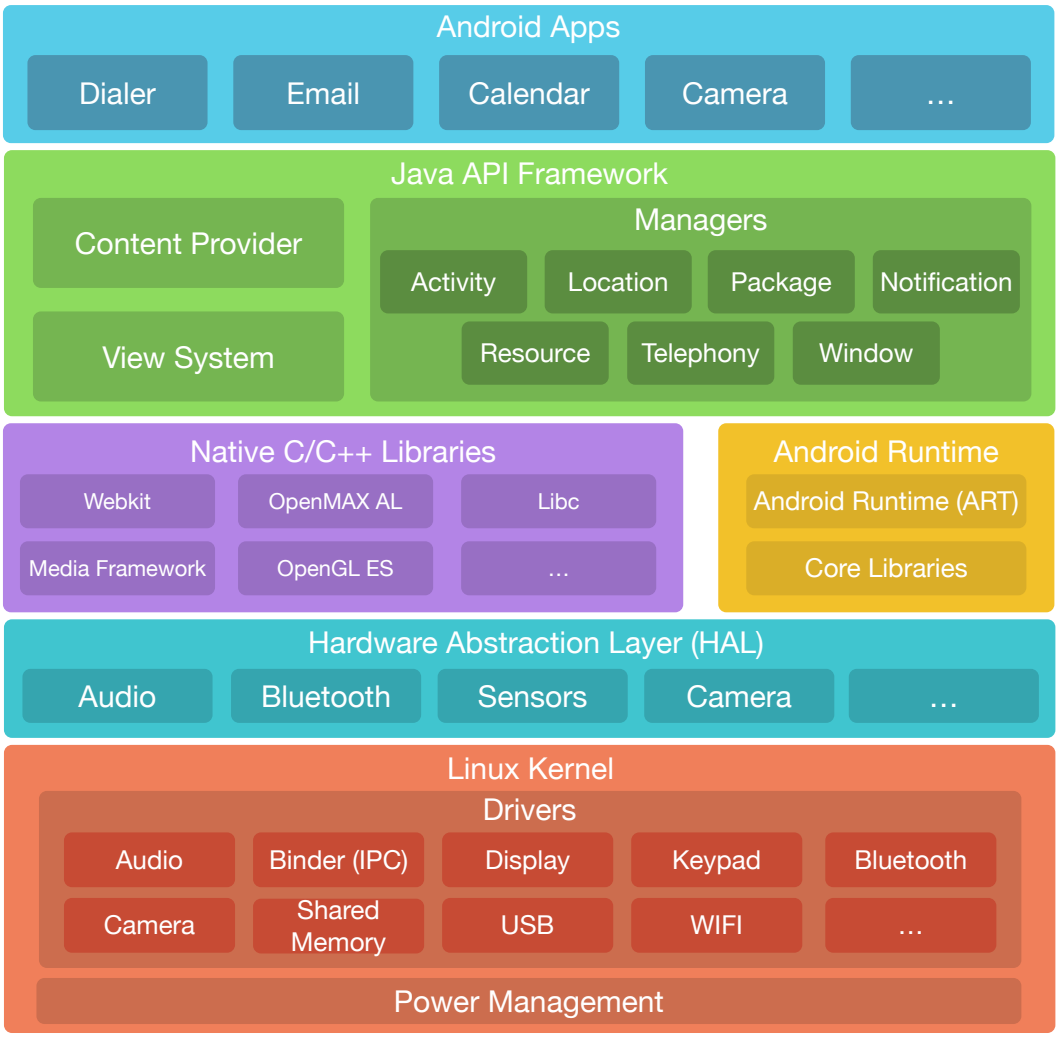


Figure 2.1: Android platform architecture [2].

using the Linux kernel, Android inherits the security feature of Linux and offers the convenience for developing hardware drivers.

Hardware Abstraction Layer (HAL). The hardware abstraction layer (HAL) provides standard interfaces for the higher-level Java API framework. It consists of multiple library modules, and each of the modules implements an interface for a specific type of hardware component. The Android system will load a

corresponding module when there is an API call for accessing a hardware component.

Android Runtime. Android Runtime (ART) provides a runtime environment for the running apps. For devices with Android version 5.0 (API level 21) or higher, each app runs in its own process and has its own ART instance. ART applies ahead-of-time (AOT) and just-in-time (JIT) compilation and optimises garbage collection (GC) compared to the former Android runtime, Dalvik.

Native C/C++ Libraries. For the purpose of low latency, native libraries written in C and C++ code are required. Multiple Android framework components (e.g., ART and HAL) are built from native libraries. The Android platform provides Java APIs to expose the functionality of these native libraries to apps.

Java API Framework. Through APIs in Java language, the entire feature-set of Android operating system can be accessed. These APIs help developers create applications by simplifying the reuse of modular system components and services.

Android Apps. The top layer of the Android platform is the Android applications including both core apps and third-party apps. The Android system comes with a set of core apps such as `Contact`, `Email` and `Calendar`. They function both as apps for users and provide key capabilities that developers can access. Depending on the purposes, users could also install third-party applications from Google Play, the official app market of Android, or other app repositories.

The focus of our research is detecting event-based races existing in Android apps. Therefore, the next part provides the background information about the applications.

2.1.2 App Components

An Android app is composed of *components*. Each of them is an entry point through which the system or a user can enter the app. There are four types of components defined by Android, each serving different purposes. The functionality of each type of component is described below.

Activity. Activities are the most popular components in Android apps. An activity corresponds to a visible screen containing different user interface (UI) elements (e.g., buttons and images) that allow the user to interact with the device and apps. An app starts from a developer-defined entry activity when opened and then reacts to user and system actions to proceed. Of the four types of components, activities lie at the heart of Android's event-driven programming framework. Through the UIs in activities, users are able to generate different events (e.g., click or swipe) to control the execution of an app.

Service. Unlike the activities that contain UI widgets in the screen, services do not have any UI and are used for handling time-consuming tasks in the background or working for remote processes. For example, a service can be leveraged to play music or download data from the Internet in the background so that the user is able to use another app at the same time. Another component, such as an activity, can start a service or bind to it for interacting.

Broadcast Receiver. The Android operating system sends broadcasts when various system events occur, such as when the system boots up or the device starts charging. An app is also able to broadcast to announce the change of its status. Targeting the broadcasts, Android allows apps to register broadcast receivers in order to respond different broadcasts.

Content Provider. The data of apps or system could be stored in different forms (e.g., SQLite database or file system). Through a unified set of APIs, content

provider offers a consistent, standard interface to data that also handles inter-process communication and secure data access.

In an Android app, there are usually several components working together via inter-component communication (ICC). Users can navigate among different components for different purposes. From the entry component, an app executes along component transition paths and different event methods of components will be invoked.

2.2 Android Concurrency

2.2.1 Processes and Threads

Inherited from the Linux kernel, Android also leverages processes and threads for concurrency purposes. We discuss how processes and threads work in an Android application below.

Processes. When an application component is launched and there is no other components running, the Android system creates a new process for the application. Other components in the same app are assigned running in the same process by default. However, Android also allows arranging different components in an app to run in separate processes. When the resources of a process are required by other processes with higher priority, the Android system will shut down the process and, consequently, the components running in the process are destroyed. For example, Android will kill a process containing activities that are no longer visible for the purpose of satisfying the requirements of another process that contains the activity shown on the screen.

Threads. A process of an Android app initially contains one thread created with the process called the *main thread*. Android does not fork a separate thread

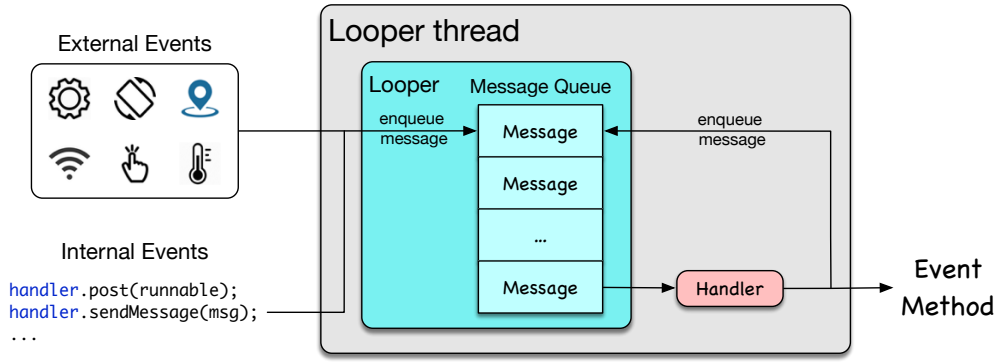


Figure 2.2: Event dispatching in Android.

for each instance of a component and all components in the same process run in the main thread. Consequently, only this thread is in charge of handling all the interactions with the UI widgets and reacting system actions. Therefore, the main thread is also known as the *UI thread*.

To ease the burden of the UI thread, Android allows the application to create background threads for time-consuming work in order to not block the main thread. The background threads cannot update the UI widgets directly but are able to access the UI thread with the event dispatching system in Android, which is discussed in the next part.

2.2.2 Event Dispatching System

As an event-driven operating system that is able to support a variety of input modalities, Android leverages an event dispatching system to orchestrate the execution of Android application. Figure 2.2 demonstrates the basic mechanic of event dispatching in Android.

Events. *Events* are the driving power of the event dispatching system and they can be generated from different sources as shown in the left part of Figure 2.2. Externally, events come from external actions such as sensor inputs (e.g.,

click and rotate) and system notifications (e.g., activity create). After an external events is generated, Android sends a corresponding message (referred to as *event messages*) to the *message queue* in a *looper thread* (explained later). Internally, Android provides a `Handler` class that allows apps to generate events using their own code. Through a `Handler` object, the app can not only directly enqueue a message (`handler.sendMessage(msg)`), but also post a `Runnable` object (`handler.post(runnable)`), which will be transformed to message by the `Handler`, to the message queue.

Looper and Looper Thread. The right part of Figure 2.2 presents the working scheme of a `Looper` object and a *looper thread*. Android provides a `Looper` class to handle the messages sent by different events. If a background thread is attached with a `Looper` object that confers the thread with event dispatching ability, it is called a *looper thread*. Each looper thread is allowed to maintain at most one `Looper` object. Inside a `Looper` object, there is a *message queue*, where all the event messages are stored. As long as a looper thread is running, its `Looper` object will continuously monitor its message queue and dispatch one message at a time following the FIFO (First-In, First-Out) principle. For each event message, it will be processed by a `Handler` object to invoke a corresponding *event method*. Once a looper thread begins to process a message, no other message process routine can occur until the event method of the message finishes its execution. Therefore, all event methods executed in the same looper thread are atomic with respect to each other.

By default, there is only one looper thread in an app process, which is the UI thread responsible for all the UI interactions and system notifications. New looper threads can be created by an app itself to handle internally generated events.

Handler. In Android, each `Handler` instance is associated with a corresponding message queue to perform operations on the related event messages. Through various APIs of the `Handler` class, apps are able to enqueue messages in different ways. An event message could be sent to a message queue with a time constraint that determines when it can be processed. The time constraint can be either an absolute timestamp (`handler.sendMessageAtTime(msg, time)`) or a delay time (`handler.sendMessageDelayed(msg, delayTime)`) with respect to the time when the the message is generated. In addition, an event message could also be directly put in the front of the message queue (`handler.sendMessageAtFrontOfQueue(msg)`).

Except for enqueueing, `Handler` is responsible for processing the messages from the message queue to invoke corresponding event methods. If a message is sent by an external event, its event method is also known as a *callback* (method). For an internally generated event message, the event method is either the `run()` method of a `Runnable` object or the `handleMessage(msg)` method of the `Handler` object itself.

2.2.3 Non-Determinism

Under the event dispatching system, the control flow of an Android app is dictated by a sequence of events generated. However, the execution order of most event methods is non-deterministic due to several reasons. The first reason is that sensor inputs and system notifications are unpredictable affecting the execution order of event methods. Second, through various APIs of the `Handler` class, an event could generate another event, whose event method runs in a different looper thread. Therefore, different event methods running on different threads also cause the non-deterministic nature of their execution order.

2.2.4 Happens-Before Relations

Although the majority of event methods are executed non-deterministically, some may establish a *happens-before relation* with others. If an event A has a happens-before relation with an event B, the event method of A will always be executed before B's event method. For example, when an Android app is started, the system always invokes a callback method `onCreate()` before any other callbacks. Therefore, the event causing the execution of `onCreate()` is considered having a happens-before relation to other events invoking other callback methods. Establishing happens-before relations between different events is essential in detecting event-based races in Android as it allows the detection tool to identify the actual non-deterministic execution order between event methods.

2.3 Event-Based Races

Because of the non-deterministic execution orders between different event methods, the event dispatching system introduces event-based races, which is the major type of concurrency bugs in Android. According to the research [51], there are four to seven times more event-based races than traditional multi-thread data races. In our thesis, we focus on detecting the races that lead to use-after-free (UAF) violations, which can cause unexpected termination of the apps and even security vulnerabilities [4]. An event-based race that causes a UAF violation happens when a pointer is dereferenced, i.e., used after it has been freed, where its use and free operations can be reached from two event methods. Due to the non-deterministic execution order between the two event methods, the use operation may be potentially executed after the free operation.

```
1. public class PortForwardListActivity extends Activity{
2.  protected TerminalBridge hostBridge = null;
3.  protected Handler updateHandler = new Handler();

4.  private ServiceConnection connection = new ServiceConnection(){
5.   public void onServiceConnected(ComponentName className, IBinder service){
6.    PortForwardListActivity.this.hostBridge = new TerminalBridge();
7.   }
8.   public void onServiceDisconnected(ComponentName className){
9.    PortForwardListActivity.this.hostBridge = null;
10.  }
11. };

12. public void onStart(){
13.  bindService(new Intent(this, TerminalManager.class), this.connection, 1);
14. }

15. public void onClick(View arg0){
16.  if(hostBridge == null) return;
17.  updateHandler.post(new Runnable(){
18.   public void run() {
19.    PortForwardListActivity.this.hostBridge.enablePortForward();
20.   }
21.  });
22. }}
```

an event-based race

Figure 2.3: An event-based race example in `ConnectBot`

Figure 2.3 presents an example of a typical event-based race in the app `ConnectBot` [3]. In the example, the field `hostBridge` is freed at line 9 that can be reached from the event method `onServiceDisconnected()` (lines 8–10) and is used at line 19 of the event method `run()` (lines 18–20). However, the execution order of the two event methods is non-deterministic, such that a UAF event-based race can happen under certain circumstances.

When the app starts the activity `PortForwardListActivity`, the event method `onStart()` (lines 12–14) is automatically invoked by the system. The statement at line 13 binds the activity to a service component and the event methods `onServiceConnected()` (lines 5-7) and `onServiceDisconnected()` are invoked by the framework under certain condition to allow activity to interact with the service. In the event method `onClick()` (lines 15–22), the conditional statement at line 16 checks whether the field `hostBridge` is null in order to prevent potential UAF violations. Unfortunately, `onClick()` posts another event,

`run()`, that will use `hostBridge` later. An event-based race will happen if an user first clicks the corresponding button to trigger `onClick()` and the event method `onServiceDisconnected()` is invoked by the system after `onClick()` but before `run()`. In that case, the field `hostBridge` is used after it has been freed, causing an unexpected termination of the app.

Chapter 3

Precise Static Happens-Before Analysis for Detecting UAF Order Violations in Android

3.1 Overview

The significant growth of multi-core smart phone devices provides unprecedented opportunities for mobile apps to perform sophisticated tasks that are comparable to software applications running on desktop/laptop computers. In order to explore the full capability of multi-core mobile phones, Android provides a rich set of APIs to support a hybrid concurrency system consisting of both traditional Java threads and asynchronous events. However, such a system introduces both intra-thread (caused by asynchronous events) and inter-thread concurrency bugs (caused by both events and Java threads), which are difficult to detect, in practice.

Unlike the case in the Java thread-based concurrency model, Android concurrency bugs often manifest themselves in the form of order violations (also known

as event-based races) [26, 31, 32]. For a pair of a pair of memory accesses, e.g., $L1 : p = null$ and $L2 : .. = p.use()$, where $L1$ in an event method $e1$ and $L2$ in another event method $e2$, if the execution order of $e1$ and $e2$ is non-deterministic (no correct happens-before relation), there is a use-after-free order violations, which is able to cause unexpected termination of an app.

It is challenging to find order violations precisely in Android apps because of the complexity of its concurrency mechanism involving both Java threads and asynchronous events. Android allows an individual thread to maintain its own `Looper` object, which makes the thread can accept asynchronous events posted from other threads. Furthermore, a thread can also be created via an asynchronous event, which can also significantly complicate the analysis of event interleavings. Therefore, reasoning the execution order between the event methods is essential yet difficult.

Most of the existing Java-based concurrency bug detection tools [18, 21, 22, 24, 25, 47, 56, 59, 60, 63, 64, 72, 73, 76, 78–80] are unaware of Android events. Simply applying these tools for detecting order violations in Android works poorly due to the complex concurrency model used for dispatching non-deterministic events. Existing efforts in detecting UAF order violations in Android mostly focus on dynamic analysis [16, 31, 51], which first collects execution traces by exercising an app at runtime through fuzzing [16, 51] or manual exploration [31]. Then an offline detection on the collected traces is performed. Due to the nature of dynamic analysis, dynamic race detection approaches suffer from limited code coverage in the presence of an unbounded number of event interleavings. In addition, these tools usually require multiple runs in order to generate more traces to enable an effective bug detection, resulting potentially in large runtime overheads.

Static detection of order violations will not suffer from the above mentioned limitations. However, static techniques for UAF detection [26, 32] are relatively unexplored due to the difficulty in modeling abstract asynchronous events under infinite event interleavings. Sierra [32] includes an event-based race detection, but does not consider inter-thread order violations. nAdroid [26] represents a recent approach to detecting order violations by converting asynchronous events into threads and then applying a traditional data race detection tool for analyzing Java programs [56]. However, nAdroid relies on coarse-grained flow- and context-insensitive event modeling, which may miss some happens-before (HB) relations among the events, causing a large number of false alarms to be issued. In addition, nAdroid may also introduce spuriously HB relations unsoundly (due to its one looper thread assumption), causing some bugs to be missed (Section 3.3).

To address the aforementioned limitations, this chapter presents Sard, a static approach to detecting UAF order violations, the most common type of races in Android [26]. Sard systematically models Android's asynchronous events to detect both intra- and inter-thread order violations. A new flow- and context-sensitive static happens-before analysis is proposed to reason about the interleavings between events in a single and/or multiple looper threads to identify precise HB relations and significantly remove spurious event interleavings. Our static happens-before relations can also be used to accelerate dynamic analysis by avoiding exercising event orders that are statically proved to be safe.

Sard performs context-sensitive analysis by distinguishing the calling contexts leading to an API call that creates or dispatches an event. Our flow-sensitive analysis precisely reasons about the control-flow execution order inside an atomic method for determining event-posting orders. In addition, nAdroid assumes that only one message queue for all events across all threads during their static modeling,

i.e., yielding spurious HB relations. In contrast, SARD's modeling is more sound as it can discover more UAFs than NADROID.

We have evaluated SARD using 27 real-world Android apps. Our results show that SARD significantly outperforms NADROID, a state-of-the-art static Android order violation detection tool, in terms of both precision (by reporting three times fewer false alarms and three more true alarms given the same set of apps used by NADROID) and efficiency (by running two orders of magnitude faster).

This chapter makes the following key contributions:

- We present a new static order violation detection approach by precisely reasoning about the happens-before relations between asynchronous events in Android apps.

- We introduce a new flow- and context-sensitive approach to modeling Android events in single and multiple message queues to reason about event interleavings.

- We evaluate our tool SARD with a set of 27 real-world Android apps. SARD significantly outperforms NADROID, a state-of-art static analysis tool for detecting UAF order violations, in terms of both precision (by reporting 1058 fewer false positives and 3 more true alarms) and efficiency (by running 175 times faster).

The rest of the chapter is organized as follows: Section 3.2 discusses the related work. Section 3.3 describes the motivation of SARD. Section 3.4 introduces the details of our approach. Section 3.5 evaluates SARD with real-world Android apps. At last, we concludes this chapter at Section 3.6.

3.2 Related Work

Detecting use-after-free races in Android is a new research area relative to traditional Java race detection. There are both static and dynamic approaches proposed.

Static Analysis for Android. The work that is the most related to our static tool Sard is nAdroid [26]. It converts asynchronous events into native Java threads and leverages the Java race detector Chord [56] to perform race detection. We will compare our tool with nAdroid in Section 3.5 and the results show that Sard outperforms nAdroid in both effectiveness and efficiency.

Recently, a static tool, Sierra [32], has been introduced for detecting *event-based* races in Android applications. Sierra applies an *action-sensitive* pointer analysis and builds happens-before relations between asynchronous events for *event-based* race detection. However, unlike Sard, Sierra handles only the *event-based* races that happen within the same thread. In addition, Sierra also ignores the contexts of events, which will induce different running threads and HB relations with others. Currently, the source code of Sierra is not available yet, so we are not able to compare our tool with it.

Some other static tools also have been proposed to detect races in Android. Asynchronizer [44] is a static refactoring tool to extract long-running operations in AsyncTask (an encapsulated thread class). However, this tool focuses on AsyncTask only and is not able to detect the races caused by asynchronous events. DEvA [62] is another static tool for detecting races in Android apps. This work is also limited for only detecting races between two callbacks without modeling their HB relations. This limitation causes the tool to suffer from significant false positives and false negatives.

Dynamic Analysis for Android. Dynamic tools [31, 51] are developed to detect races in Android at runtime. Their approaches first collect execution traces,

which are generated by running Android apps on devices with their customized ROM. They then perform an off-line HB relations analysis to detect races on the collected traces. EventRacer [16] uses an novel off-line analysis algorithm to improve the scalability and precision of the previous approaches. Recently, the authors of [30, 50] also introduce new approaches to building the HB relations for Android apps based on execution traces. While reporting fewer false positives, dynamic tools suffers from limited code coverage and extra runtime overheads.

Race Detection for Java. There are quite a few existing approaches that can detect data races in traditional Java programs. There are static tools, based on, for example, locksets [21, 56, 59, 73], type systems [18, 24], May-Happen-in-Parallel (MHP) relations [34, 84] and model checking [29, 60]. There are also dynamic analysis tools [22, 25, 35, 47, 63, 64, 72, 75, 76, 78–80]. These tools are not aware of Android's hybrid concurrency system, which includes not only threads but also asynchronous events, and the interleavings between different events and threads in Android apps are more complicated. Therefore, traditional tools for Java programs either do not scale or produce many false positives.

3.3 A Motivating Example

This section demonstrates how Sard precisely extracts the intra- and inter-thread happens-before relations that are missed conservatively or inferred unsoundly (i.e., incorrectly) by nAdroid [26].

In the code of Figure 3.1(a), lines 16 and 17 in `onCreate()` post two events `a.run()` (blue) and `b.run()` (green) to the message queue with their method bodies containing a field use $p.use()$ and a field nullifying statement $p = null$, respectively. nAdroid is unable to know that `b.run()` is always dispatched after `a.run()` due

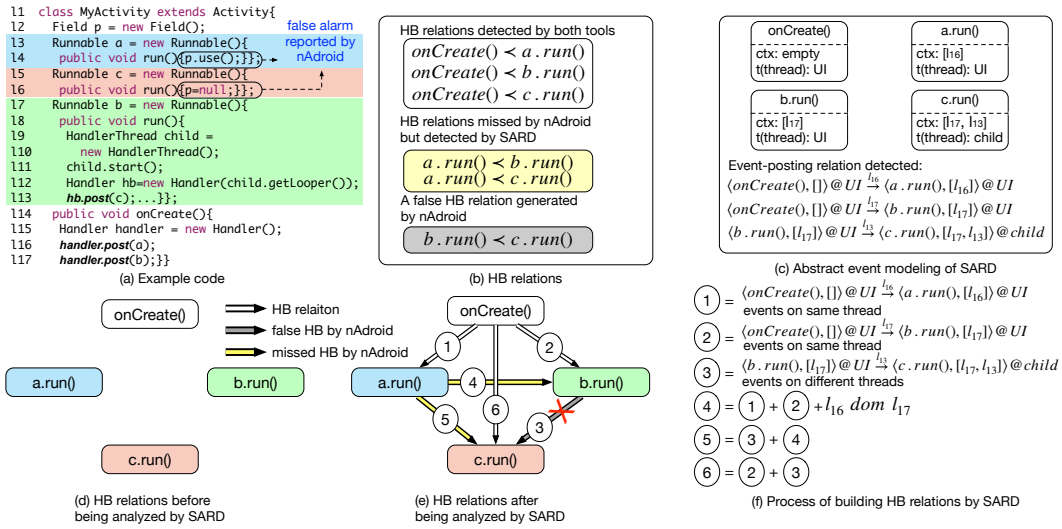


Figure 3.1: A motivating example. $\langle e_x, c_x\rangle@t_x \xrightarrow{s} \langle e_y, c_y\rangle@t_y$ **means that event** x **with context** c_x **in the message queue of thread** t_x **posts event** y **with context** c_y **to the message queue at thread** t_y **with statement** s**. If** s **and** s' **are two statements that appear in the inter-procedural CFG starting from a common method, then** s *dom* s' **signifies that** s **dominates** s' **in the CFG.**

to the control-flow execution order (lines 16 and 17) inside the atomic method `onCreate()` for posting the two events. `b.run()` posts a new event `c.run()` (red) at line 13 to the message queue of a parallel thread `child` created at lines 9-11. NAdroid conservatively assumes that `c.run()` can happen in parallel with `a.run()` without performing any analysis and reports a false order violation. However, `c.run()` in the `child` thread is posted via the callsite at line 13 in `b.run()`, which must be executed after `a.run()`. On the contrary, Sard is able to infer this strict inter-thread HB relation from `a.run()` to `c.run()` by analyzing the program control-flow from line 16 to line 17, thereby eliminating the false alarm reported by nAdroid.

As shown in Figure 3.1(b), Sard can precisely identify the five HB relations, $onCreate() \prec a.run()$, $onCreate() \prec b.run()$, $onCreate() \prec c.run()$, $a.run() \prec b.run()$, and $a.run() \prec c.run()$, among which $a.run() \prec b.run()$ and $a.run() \prec$

$c.run()$ are missed by NAdroid (in yellow) and $b.run() \prec c.run()$ is incorrectly introduced (in grey).

3.3.1 Existing Techniques

NAdroid may miss some HB relations conservatively due to its imprecise flow- and context-insensitive modeling for the event interleavings and introduce some spurious HB relations unsoundly due to an unsound assumption about the existence of only one message queue for all events across all the threads.

NAdroid first converts all the asynchronous events into traditional threads in order to leverage Chord, a traditional race detector for Java [56]. NAdroid builds a harness main method for an Android app and creates artificial threads to invoke methods of asynchronous events in the harness main. After running Chord, NAdroid then prunes the results obtained with its own modeling of HB relations, which is conservative as it assumes only that `onCreate()` happens before all other Android events (including callbacks and runnable events) of an activity and `onDestroy()` happens after all other callbacks. As shown in Figure 3.1(b), `onCreate()` happens before `a.run()`, `b.run()` and `c.run()` as expected but the two HB relations (in yellow) are missed.

In addition, as highlighted in grey, NAdroid is also unsound as it may introduce spurious HB relations between two events in two different queues of two parallel threads. NAdroid infers an incorrect HB relation between `b.run()` and `c.run()`, since NAdroid assumes a single unique message queue. However, the two events actually reside in the parallel message queues in two different threads, i.e., UI and child threads.

3.3.2 SARD

SARD performs a flow- and context-sensitive analysis that correctly handles the calling contexts and program control-flows when analyzing the event-posting and event-dispatching related Android APIs. SARD precisely models an abstract event via a calling context under an abstract thread.

SARD first performs a pre-analysis to model the four abstract events with its corresponding context information given in Figure 3.1(c). The context of each abstract event is a stack represented by a sequence of call statements leading to that event. For example, the context of an abstract event `c.run()` is represented as $[l_{17}, l_{13}]$ created via invocations l_{17} and l_{13}.

SARD provides fine-grained modeling of events by distinguishing message queues in different threads. Abstract threads including the default UI thread and child threads at a thread creating site (e.g., l_9 - l_{10}) are modeled. SARD determines the thread where an event resides in by analyzing the Android `Handler` object. For example, event `c.run()` is in the `child` thread since `c.run()` is posted to `child` via the `handler` created at line l_{12}. Figure 3.1(c) gives the four abstract events and their corresponding contexts, from which we can directly obtain their event-posting relations. We write $\langle onCreate(), []\rangle @UI \xrightarrow{l_{16}} \langle a.run(), [l_{16}]\rangle @UI$ to indicate that `onCreate()` in the message queue of the UI thread posts `a.run()` to the UI's message queue under the invoking statement at line l_{16}. Similarly, the other two event-posting relations are $\langle onCreate(), []\rangle @UI \xrightarrow{l_{17}} \langle b.run(), [l_{17}]\rangle @UI$ and $\langle b.run(), [l_{17}]\rangle @UI \xrightarrow{l_{13}} \langle c.run(), [l_{17}, l_{13}]\rangle @child$.

Given the abstract event modeling and event-posting relations, we will start to infer the HB relations (Figure 3.1(f)) for detecting UAF order violations. Initially, there are no HB relations for the four events, implying that any two events may happen in parallel as illustrated in Figure 3.1(d).

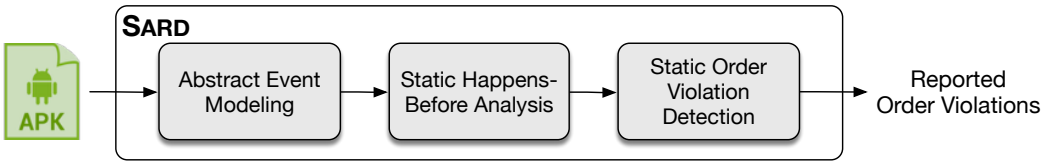


Figure 3.2: Workflow of Sard.

From the first two event-posting relations in Figure 3.1(c), we obtain $\langle onCreate(), []\rangle \prec \langle a.run(), [l_{16}]\rangle$ and $\langle onCreate(), []\rangle \prec \langle b.run(), [l_{17}]\rangle$, which are ① and ② respectively. Because SARD's modeling can distinguish the message queues of different threads (e.g., `b.run()` and `c.run()` associated with two different threads UI and *child* as identified by ③ in Figure 3.1(f)), SARD can infer a may-happen-in-parallel relation instead of an unsound HB relation reported by NADROID, which assumes a single message queue for all threads.

By considering the control-flows, SARD determines that `a.run()` and `b.run()` are posted in the order as denoted by the dominance relation l_{16} *dom* l_{17}. With this flow-sensitive information together with ① and ②, SARD can easily infer ④, i.e., `a.run()` happens before `b.run()`.

We can obtain ⑤, i.e., `a.run()` happens before `c.run()` since ④ indicates that `a.run()` is executed before `b.run()`, which has posted `c.run()` for execution (③).

Since `onCreate()` happens before `b.run()`, as identified by ②, the HB relation ⑥ holds transitively for any other events (e.g., `c.run()`) posted by `b.run()` (③).

Finally, based on the HB relations in Figure 3.1(e), SARD can prove the absence of UAF violations in this example because the field use at l_4 in `a.run()` always happens before the null pointer assignment at l_6 in `c.run()` according to ⑤, which is missed by NADROID, causing a false alarm.

3.4 Approach

In this section, we introduce the approach used in SARD, as shown in Figure 3.2. SARD first models all the context-sensitive abstract events and identifies the event-posting relations. Then, SARD performs a flow- and context-sensitive analysis to infer the HB relations for all the abstract events. Finally, SARD detects order violations between two events based on their HB relations and the field usage in the two events. SARD applies a light-weight feasible path analysis to further prune out false alarms by identifying spurious UAF pairs.

3.4.1 Abstract Event Modeling

Abstract events are modeled context-sensitively to distinguish events under different calling contexts. A context-sensitive abstract event is denoted by $\langle e, c\rangle$, where the context $c \in \mathbb{C}$ of an event e is a stack represented by a sequence of call statements leading to the executing method of e. An abstract thread t is a thread where an abstract event resides in. In SARD, we use the allocation sites of thread objects to represent abstract threads. If an event runs in the UI thread that does not have allocation sites in application code, we use UI to represent its abstract thread. We write $\langle e, c\rangle @t$ to indicate that a context-sensitive event e runs in an abstract thread t under context c.

Each abstract event is modeled based on a sequence of event-creating API calls $s \in \mathbb{I}$. We have systematically modeled 22 APIs falling into three categories: (1) `handler-based` APIs, (2) `activity-based` APIs, and (3) `thread-based` APIs. In Figure 3.3, the left column gives the code examples for the three categories.

The API methods in the first category are the methods in class `Handler`, an internal class in Android. Figure 3.3(a) gives an example to demonstrate that

Three Event-Creating APIs	Abstract Event $\langle e, c \rangle @ t$

handler-based
1 Runnable a = new Runnable(){
2 public void run(){...}};
3 HandlerThread child =
new HandlerThread();
4 child.start();
5 Handler handler1 =
new Handler(child.getLooper());
6 handler1.post(a);
7 Handler handler2 = new Handler();
8 handler2.post(a);
	e: a.run()
	c: [line 6]
	t: child(line 3)
	$\langle a.run(), [l_6] \rangle @ child$
	
	e: a.run()
	c: [line 8]
	t: UI thread(null)
	$\langle a.run(), [l_8] \rangle @ UI$

(a) A handler-based API

activity-based
1 Runnable a = new Runnable(){
2 public void run(){...}};
3 public void foo(){
4 activity.runOnUiThread(a);}
	e: a.run()
	c: [line 4]
	t: UI thread(null)
	$\langle a.run(), [l_4] \rangle @ UI$

(b) An activity-based API

thread-based
1 Runnable a = new Runnable(){
2 public void run(){...}};
3 public void foo(){
4 Thread child = new Thread(a);
5 child.start();}
	e: a.run()
	c: [line 5]
	t: child(line 4)
	$\langle a.run(), [l_5] \rangle @ child$

(c) A thread-based API

Figure 3.3: Modeling abstract events based on API calls.

two abstract events created under different contexts running in their corresponding threads via a `handler-based` API. At line 6 (line 8), `handler1.post(a)` (`handler2.post(a)`) uses a handler object to post the runnable event `a.run()` to the message queue of the child (UI) thread.

$$\boxed{m\text{: the containing method of a call statement } s}$$

$$\frac{e \text{ is a callback method}}{\langle e, \emptyset \rangle @UI} \quad \text{[C-CALLBACK]}$$

$$\frac{\begin{array}{c} s : \texttt{a handler-based API call} \quad e = \texttt{getTgt}(s) \\ \langle m, c \rangle \quad c' = c.append(s) \quad t = \texttt{getThread}(s) \end{array}}{\langle e, c' \rangle @t} \quad \text{[C-POST]}$$

$$\frac{\begin{array}{c} s : \texttt{an activity-based API call} \quad e = \texttt{getTgt}(s) \\ \langle m, c \rangle \quad c' = c.append(s) \end{array}}{\langle e, c' \rangle @UI} \quad \text{[C-UI]}$$

$$\frac{\begin{array}{c} s : \texttt{an thread-based API call} \quad e = \texttt{getTgt}(s) \\ \langle m, c \rangle \quad c' = c.append(s) \quad t = \texttt{getAllocSite}(s) \end{array}}{\langle e, c' \rangle @t} \quad \text{[C-THREAD]}$$

$$\frac{s \notin \mathbb{I} : m' \text{ is invoked} \quad \langle m, c \rangle \quad c' = c.append(s)}{\langle m', c' \rangle} \quad \text{[C-CONTEXT]}$$

Figure 3.4: Rules for modeling abstract events.

For the second category, one can create an event via `runOnUiThread(runnable)` provided by Android's `Activity` class and its subclasses (e.g., `ListActivity`). All events created are posted only to the message queue of the UI thread. For example, at line 4 in Figure 3.3(b), `activity.runOnUiThread(a)` posts a runnable object to UI's message queue.

The last category contains the API methods that fork a traditional Java thread. For the statement at line 5 in Figure 3.3(c), it is treated as posting an asynchronous event to a special thread `child` whose queue only contains this event.

Figure 3.4 gives the rules to model context-sensitive abstract events by handling the calls of these three types of APIs. Rule [C-CALLBACK] builds an abstract event for every Android callback method, which runs in the UI thread. For every callback event e, its context is $c = \emptyset$, since it is created in the Android framework but not through some event-creating APIs in the application code.

If a call statement s invokes a `handler-based` API, we apply [C-POST] to model its corresponding abstract event. `getTgt`(s) returns the corresponding event posted by s and `getThread`(s) retrieves the abstract thread where the event runs. With event e, context c' and abstract thread t, we create a context-sensitive event $\langle e, c' \rangle @t$. Figure 3.3(a) gives two context-sensitive events created by this rule, i.e., $\langle a.run(), [l_6] \rangle @child$ and $\langle a.run(), [l_8] \rangle @UI$. For example, if s is $handler1.post(a)$ in Figure 3.3(a), then `getTgt`(s) returns `a.run()` and `getThread`(s) returns `child`.

For a call s (e.g., `activity.runOnUiThread(a)` in Figure 3.3(b)) that invokes an `activity-based` API method, we only create an event running in the UI thread with its corresponding context following [C-UI]. Figure 3.3(b) shows an example for creating a context-sensitive event $\langle a.run(), [l_4] \rangle @UI$.

Rule [C-THREAD] is applied to build an abstract event if s is a `thread-based` API call (e.g., `child.start()` in Figure 3.3(c)). The abstract thread t is modeled by `getAllocSite(s)`, which finds the thread allocation site, where the thread object at s is created. By applying this rule, we extract a context-sensitive event $\langle a.run(), [l_5] \rangle @child$ as illustrated in Figure 3.3(c).

For a call statement s that does not invoke any event-posting API denoted by $s \notin \mathbb{I}$, we apply [C-CONTEXT] to build new contexts for method m' invoked by s. Based on the context c of method m, which is the containing method of s, a new context c' is created by appending s to c.

$$\frac{\langle e_x, c_x\rangle @ t_x \xrightarrow{s} \langle e_y, c_y\rangle @ t_y \quad t_x = t_y}{\langle e_x, c_x\rangle \prec \langle e_y, c_y\rangle} \text{[INTRA-POST]}$$

$$\frac{\begin{array}{c}\langle e_x, c_x\rangle @ t_x \xrightarrow{s} \langle e_y, c_y\rangle @ t_y \\ \langle e_x, c_x\rangle @ t_x \xrightarrow{s'} \langle e_z, c_z\rangle @ t_z \\ t_y = t_z \quad s \ dom \ s'\end{array}}{\langle e_y, c_y\rangle \prec \langle e_z, c_z\rangle} \text{[INTRA-SAME]}$$

$$\frac{\begin{array}{c}\langle e_x, c_x\rangle @ t_x \xrightarrow{s} \langle e_y, c_y\rangle @ t_y \\ \langle e_z, c_z\rangle @ t_z \xrightarrow{s'} \langle e_w, c_w\rangle @ t_w \\ \langle e_x, c_x\rangle \prec \langle e_z, c_z\rangle \quad t_y = t_w\end{array}}{\langle e_y, c_y\rangle \prec \langle e_w, c_w\rangle} \text{[INTRA-DIFF]}$$

$$\frac{\begin{array}{c}e_x, e_z \ are \ callback \ events \\ \langle e_x, c_x\rangle @ t_x \xrightarrow{s} \langle e_y, c_y\rangle @ t_y \\ t_x = t_y \quad \langle e_x, c_x\rangle \prec \langle e_z, c_z\rangle\end{array}}{\langle e_y, c_y\rangle \prec \langle e_z, c_z\rangle} \text{[INTRA-INFER]}$$

$$\frac{\begin{array}{c}\langle e_y, c_y\rangle @ t_y \xrightarrow{s} \langle e_z, c_z\rangle @ t_z \\ \langle e_x, c_x\rangle \prec \langle e_y, c_y\rangle\end{array}}{\langle e_x, c_x\rangle \prec \langle e_z, c_z\rangle} \text{[COMBO]}$$

Figure 3.5: Rules for building happens-before relations.

3.4.2 Static Happens-Before Analysis

Once all the abstract events have been modeled, we construct an event-posting relation between different events. For an invoking statement s of event method e', if its containing method can be reached in the call graph from event method e without passing through other event methods, we write $\langle e, c\rangle @t \xrightarrow{s} \langle e', c'\rangle @t'$ to represent the fact that event e in abstract thread t under context c posts e' to the message queue of thread t' under context c' at statement s.

Given all the abstract events, SARD builds their HB relations to identify all safe use-free pairs $\langle s_{use}, s_{free}\rangle$. This ensures that a field use statement s_{use} in one event always happens before a free statement s_{free} in another event.

An abstract event can be an Android callback event or a normal event (built via the three types of API calls in the application code). We first build the HB relations between Android's callback events running on the UI thread. When creating an Android Activity or a Service component, `onCreate()` is invoked before any other callback events. Therefore, we create the HB relations from `onCreate()` to every other callback event. Similarly, `onDestroy()` happens after all other callback events, since it is the last to be invoked when exiting an Android component.

Next, SARD performs a flow- and context-sensitive analysis to build the HB relations (1) between two normal events or (2) between a callback event and a normal event following the rules in Figure 3.5. We use $\langle e_x, c_x\rangle \prec \langle e_y, c_y\rangle$ to denote an HB relation from $\langle e_x, c_x\rangle$ to $\langle e_y, c_y\rangle$, where event e_x under context c_x always happens before e_y under context c_y.

[INTRA-POST] extracts the HB relations between two events that have event-posting relations. For an event $\langle e_y, c_y\rangle$ in thread t_y posted by event $\langle e_x, c_x\rangle$ in thread t_x, where $t_x = t_y$, we have $\langle e_x, c_x\rangle \prec \langle e_y, c_y\rangle$. Let us revisit the example in Figure 3.1, `onCreate()` happens before both `a.run()` and `b.run()` based on this

rule. Due to the precision in our model, `b.run()` does not happen before `c.run()`, because these two events will be running in two different threads.

[INTRA-SAME] defines the HB relations between two events posted by the same event $\langle e_x, c_x \rangle$ by considering the control flow information between the two events. In this rule, $\langle e_y, c_y \rangle$ and $\langle e_z, c_z \rangle$ are both posted by $\langle e_x, c_x \rangle$ at statements s and s', respectively. We write s *dom* s' to indicate that s dominates s' in the inter-procedural CFG starting from $\langle e_x, c_x \rangle$, i.e., every path from the method entry of event $\langle e_x, c_x \rangle$ to s' must go through s. Based on this information and the FIFO policy for an message queue, we know that $\langle e_y, c_y \rangle$ is posted before $\langle e_z, c_z \rangle$. If $\langle e_y, c_y \rangle$ and $\langle e_z, c_z \rangle$ are in the same thread, we know that $\langle e_y, c_y \rangle$ must happen before $\langle e_z, c_z \rangle$. In Figure 3.1, we find that `a.run()` happens before `b.run()` by applying this rule.

[INTRA-DIFF] builds the HB relations for two events $\langle e_y, c_y \rangle$ and $\langle e_w, c_w \rangle$, which are posted by two different events $\langle e_x, c_x \rangle$ and $\langle e_z, c_z \rangle$, respectively. We can obtain that $\langle e_y, c_y \rangle$ happens before $\langle e_w, c_w \rangle$ if (1) $\langle e_y, c_y \rangle$ and $\langle e_w, c_w \rangle$ are in the same abstract thread, and (2) $\langle e_x, c_x \rangle$ happens before $\langle e_z, c_z \rangle$.

[INTRA-INFER] is used to model the HB relations between a callback event and a normal event posted by a callback event. We have this rule based on the Android event-driven mechanism by which the UI thread can only execute one event at a time and no other callback events (e.g., `onClick()`) can be posted to UI thread's message queue if an existing event on the UI is executing. In this rule, we can infer $\langle e_y, c_y \rangle \prec \langle e_z, c_z \rangle$ if there exists a callback event $\langle e_x, c_x \rangle$ such that (1) $\langle e_x, c_x \rangle \prec \langle e_z, c_z \rangle$ holds, i.e., the callback event $\langle e_z, c_z \rangle$ can only be posted into UI's queue after executing $\langle e_x, c_x \rangle$ and (2) $\langle e_x, c_x \rangle$ has already posted $\langle e_y, c_y \rangle$ into UI's even queue during its execution. Thus, $\langle e_y, c_y \rangle$ always happens before $\langle e_z, c_z \rangle$.

Given an HB relation $\langle e_x, c_x \rangle \prec \langle e_y, c_y \rangle$, we can easily establish $\langle e_x, c_x \rangle \prec \langle e_z, c_z \rangle$ transitively if $\langle e_z, c_z \rangle$ is posted by $\langle e_y, c_y \rangle$. Therefore, [COMBO] can yield intra-thread or inter-thread HB relations, where $\langle e_x, c_x \rangle$ and $\langle e_z, c_z \rangle$ can be in either the same thread or two different threads. In Figure 3.1, `onCreate()` and `a.run()` both happen before `c.run()` based on this rule.

3.4.3 Static Order Violation Detection

After having obtained the HB relations for all the abstract events, we can detect UAF order violations. For every event, we collect its field usage operations, i.e., field use and object free statements. SARD regards each statement that dereferences a field as a field use s_{use} and a statement that sets a field to null as a field free s_{free}. With the help of the alias analysis in [71], we collect the set $\mathbb{P}_{\text{all}}$ of all candidate pairs $\langle s_{use}, s_{free} \rangle$, where two operations that access the same object and the two field usage statements s_{use} and s_{free} are from different events. Let $\mathbb{P}$ be the set of all the UAF pairs detected by SARD:

$$\mathbb{P} = \{\langle s_{use}, s_{free} \rangle \in \mathbb{P}_{\text{all}} \mid s_{use} \not\prec s_{free}, s_{free} \Rightarrow s_{use}\}$$

where s_{use} does not happen before s_{free}, and $s_{free} \Rightarrow s_{use}$ denotes a *UAF-feasible* path from s_{free} to s_{use}, such that (1) the path is control-flow feasible from s_{free} to s_{use}, and (2) there is no assignment to initialize the underlying field object between s_{free} to s_{use} along this path.

SARD performs a light-weight path-sensitive analysis by analyzing the immediate branch conditions of s_{use} and s_{free}, where the two events s_{use} and s_{free} run on the same thread.

```
1 public void onCreate(){
2  if(field != null)
3   field.use();}
```

(a)

```
1 public void onCreate(){
2  field = new Field();
3  field.use();}
```

(b)

```
1 public void onCreate(){
2  field = getField();
3  field.use();}
```

(c)

```
1 public void onCreate(){
2  field = null;
3  field = new Field();}
```

(d)

```
1 public void onCreate(){
2  field = null;
3  field = getField();
```

(e)

```
1 public void onCreate(){
2  field = null;
3  finish();}
```

(f)

Figure 3.6: Commonly occurring infeasible paths.

Figure 3.6 gives six common infeasible paths. Figure 3.6(a) demonstrates the situations when a null check happens before a field use statement. With such a check, the use statement will never be executed after a field free statement.

Figures 3.6(b) and (c) show the scenarios when a field assignment statement is executed before a field use statement. In SARD, an assignment statement can be either a statement directly assigning a new value to the field or indirectly via Android's system methods (e.g., `activity.getIntent()`). We assume that assigning a value to a field by invoking any system method will initialize the field. Figures 3.6(d) and (e) depict the occurrence of an assignment statement immediately after a free statement. These four examples illustrate infeasible paths that can never trigger any UAF order violation due to the fact that the field object is initialized before any use.

In Android, `finish()` can be invoked to terminate an Android component. Figure 3.6(f) shows that if there is an invocation of `activity.finish()` after a

field free statement, no other events in this component can be executed, resulting in an infeasible path from s_{free} to s_{use}.

3.5 Evaluation

The objective of our evaluation is to demonstrate that SARD can effectively detect both intra- and inter-thread UAF order violations with low false alarms and high efficiency in real-world Android apps. For the same set of apps used, SARD significantly outperforms NADROID [26], a state-of-the-art static analysis tool, in terms of both efficiency (by running 175 times faster) and precision (by reporting three times fewer false alarms and identifying 3 more true alarms). In addition, SARD also achieves a false negative rate that is two times lower than NADROID on the apps with the ground truth about the UAF order violations present.

3.5.1 Implementation

SARD is built upon FLOWDROID [14], a static taint analysis for Android apps. We use FLOWDROID to decompile an Android application and then obtain all callback methods of the application. FLOWDROID uses the SPARK [38] pointer analysis in SOOT [71] framework to construct a call graph. SARD uses the call graph and the alias information provided by FLOWDROID and SPARK for our field usage analysis to support our static happens-before analysis.

3.5.2 Methodology

In order to fairly compare SARD with NADROID, we use all the 27 real-world Android apps also used in NADROID. These applications exhibit a wide range of event usage patterns through a wide variety of event-creating APIs. Since our

approach adopts a more precise flow- and context-sensitive modeling of the Android concurrency system, Sard can successfully identify HB relations that are missed by nAdroid, thereby eliminating spurious UAF violation pairs (Section 3.4.2). Sard also applies a light-weight path-sensitive analysis to discover UAF infeasible paths to further remove more false alarms (Section 3.4.3).

To further validate the effectiveness of Sard, we use 8 Android apps that have been manually injected with real UAF order violations to demonstrate that Sard can find UAF order violations in a low false negative rate.

Our experiments are conducted on a quad-core i5-6500 3.2GHz machine with 16GB RAM running Ubuntu 16.04 LTS. The analysis time of every app is the average of three runs. Our evaluation answers the following research questions (RQs):

- **RQ1.** Can Sard effectively and efficiently detect UAF order violations in real-world Android apps?
- **RQ2.** Does Sard perform better than nAdroid, a state-of-the-art static tool in detecting UAF order violations?
- **RQ3.** Can Sard recall more manually injected UAF order violations than nAdroid?

3.5.3 RQ1. Effectiveness and Efficiency of Sard

In this section, we evaluate the overall performance and the effectiveness of Sard in removing false alarms.

Table 3.1 illustrates the effectiveness of Sard in analyzing 27 large real-world Android apps, consisting of 537K lines of Java code in total. This table is parti-

Table 3.1: The effectiveness and efficiency of Sard in analyzing 27 real-world Android apps.

App Name	LOC	#Potential Violations	#Feasible Violations	#False Positives Eliminated by HB relations							#Violations Reported	Time (secs)
				[Callback Relation]	[intra-post]	[intra-same]	[intra-diff]	[intra-infer]	[combo]	Total		
SoundRecorder	1194	9	9	9	0	0	0	0	0	9	0	0.96
Swiftnotes	1571	0	0	0	0	0	0	0	0	0	0	1.02
Photoaffix	1924	379	66	38	14	0	0	1	8	61	5	17.50
MLManager	2073	64	0	0	0	0	0	0	0	0	0	11.79
InstaMaterial	2248	102	3	0	0	0	3	0	0	3	0	2.46
Tomdroid	2372	0	0	0	0	0	0	0	0	0	0	1.66
ToDoList	2637	44	10	10	0	0	0	0	0	10	0	4.47
SGT puzzle	2944	585	7	0	7	0	0	0	0	7	0	1.14
Aard	3684	718	121	0	28	0	18	0	15	61	60 (8)	15.44
Clipstack	3948	0	0	0	0	0	0	0	0	0	0	1.99
KissLauncher	5210	0	0	0	0	0	0	0	0	0	0	1.42
Zxing	6453	113	15	4	1	0	0	5	5	15	0	9.34
DashClock	10147	25	9	0	2	1	2	0	4	9	0	2.49
Dns66	10423	11	7	7	0	0	0	0	0	7	0	12.60
Music	10518	22633	3545	388	379	0	610	1159	955	3491	54	28.02
CleanMaster	11014	19	17	0	0	0	7	0	0	7	10	15.98
Omninotes	13720	2395	116	0	44	12	6	0	54	116	0	30.29
Solitair	15478	18	0	0	0	0	0	0	0	0	0	7.82
MyTracks_1	27080	2458	467	65	8	0	0	2	0	75	392 (45)	12.07
Mms	27578	2657	1066	63	4	0	0	4	78	149	917	20.91
Browser	30675	10647	788	271	84	3	118	158	147	781	7	76.26
ConnectBot	32645	150	44	30	0	0	0	0	1	31	13 (13)	80.47
MyTracks_2	37031	10894	938	55	203	152	4	3	0	417	521 (52)	62.56
MiMangaNu	37827	6	0	0	0	0	0	0	0	0	0	5.21
QKSms	56082	493	45	0	0	0	0	0	1	1	44 (28)	20.90
K9-Mail	78437	2413	255	116	41	0	0	0	20	177	78	63.81
Firefox	102658	20721	1134	6	385	7	8	0	48	454	680 (1)	66.63
Total	537571	77554	8662	1062	1200	175	776	1332	1336	5881	2781 (147)	575.19

tioned into five parts (separated by "||"). The first part gives the information about the apps used in our evaluation, including their names and lines of code (LOC).

The second part of Table 3.1 gives the number of *raw* potential order violations generated by SARD, which will be gradually scrutinized by our precise HB analysis. A candidate violation pair generated by SARD has two parts: (1) a pair of conflict operations (i.e., a free statement in event e_x and a use statement in event e_y) on the same field object, and (2) the corresponding contexts of the two events $\langle e_x, c_x \rangle$ and $\langle e_y, c_y \rangle$.

The third column of Table 3.1 gives the afore-mentioned potential order violations in each app without applying any refinement. The fourth column gives the remaining number of violation pairs after we have applied our feasible path analysis as discussed in Section 3.4.3.

The third part in Table 3.1 ranges from Column 5 to Column 11. The first six columns, respectively, illustrate the capabilities of each rule of our model (Figure 3.5) in eliminating false positives by working together to extract the HB relations in an app. Furthermore, the last column gives the total number of false positives removed by SARD's precise HB relations. For all the apps, the HB relations between only Android callback events help us eliminate a total of 1062 false pairs. Furthermore, SARD's HB relation rules, [INTRA-POST], [INTRA-SAME], [INTRA-DIFF], [INTRA-INFER] and [COMBO] are effective in removing 1200, 175, 776, 1332 and 1336 false pairs, respectively. In total, 5881 false alarms have been removed from the original 8662 feasible violations.

Finally, the fourth part gives the number of UAF order violations reported by SARD for each app, where the numbers in brackets are the true violations checked manually. In total, 2781 violations are reported by SARD and 147 of them are true UAF order violations based on manual inspection.

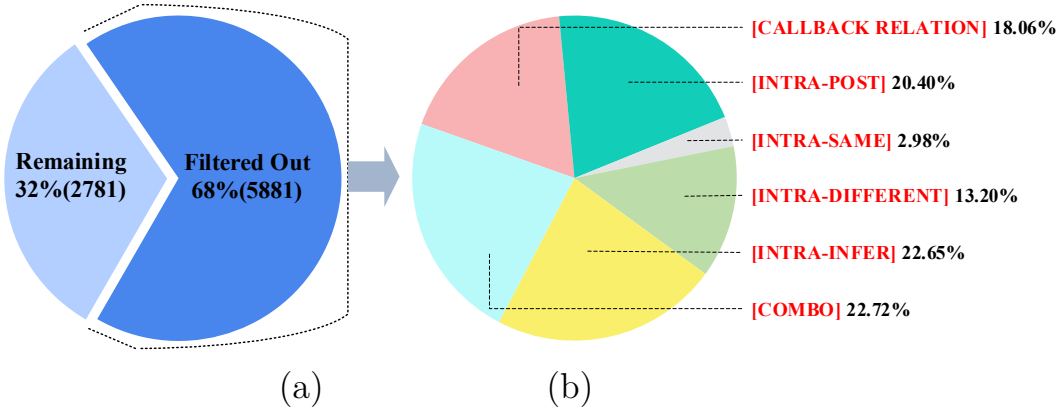


Figure 3.7: Pie charts for visualizing the data in Table 3.1.

Figure 3.7 visualizes the data in Table 3.1. The deep blue slice in Figure 3.7(a) represents 68% of false positives removed by SARD. This is further decomposed into six slices in the pie chart in Figure 3.7(b) with each slice representing the percentage of false positives removed by each rule in our HB analysis.

Table 3.1 also illustrates the analysis time (including call graph construction, our abstract event modeling, static HB analysis and static order violation detection) spent on each app in the last column. For the 27 Android apps, SARD spends only 21.3 seconds for each app on average and 575.19 seconds in total. This confirms that SARD is efficient in analyzing the real-world Android apps.

3.5.4 RQ2. SARD vs. NADROID: Effectiveness and Efficiency

In this section, we compare SARD with NADROID in finding UAF order violations in real-world Android apps. Since both do not find any violations in the same 11 apps listed in Table 3.1, the remaining 16 apps are used in this experiment. As the order violations reported by NADROID are not differentiated by the contexts of events that have field usage operations, we merge the contexts of events among our reported violation pairs in Table 3.1 and then compare with NADROID. For effectiveness,

Table 3.2: Comparing Sard and nAdroid in effectiveness and efficiency. Sard ∩ nAdroid represents the number of UAF order violations that are reported by both Sard and nAdroid. Sard \ nAdroid is the number of UAF order violations reported by Sard alone, and nAdroid \ Sard is the number of UAF order violations reported by nAdroid alone.

App Name	#Violations Reported			Sard \ nAdroid			nAdroid \ Sard			Time (secs)	
	nAdroid	Sard	nAdroid ∩ Sard	#True Positives	#False Positives	Total	#True Positives	#False Positives	Total	Sard	nAdroid
PhotoAffix	4	5	0	0	5	5	0	4	4	17.50	502.22
Aard	48	13	13	0	0	0	0	35	35	15.44	4367.66
KissLauncher	36	0	0	0	0	0	0	36	36	1.42	586.93
Zxing	2	0	0	0	0	0	0	2	2	9.34	3828.89
Dns66	13	0	0	0	0	0	0	13	13	12.60	703.58
Music	207	48	38	0	10	10	0	169	169	28.02	664.09
CleanMaster	0	6	0	0	6	6	0	0	0	15.98	8496.46
Solitaire	1	0	0	0	0	0	0	1	1	7.82	388.68
MyTracks_1	80	52	48	2	2	4	0	32	32	12.07	2708.79
Mms	312	182	120	0	62	62	0	192	192	20.91	1119.86
Browser	0	7	0	0	7	7	0	0	0	76.26	3338.98
ConnectBot	13	13	13	0	0	0	0	0	0	80.47	1265.98
MyTracks_2	71	74	56	1	17	18	0	15	15	62.56	14983.00
QKSMS	19	11	11	0	0	0	0	8	8	20.90	12603.63
K-9 Mail	336	76	20	0	56	20	0	316	316	63.81	25477.80
FireFox	468	68	31	0	37	57	0	437	437	66.63	11095.86
Total	1610	555	350	3	202	205	0	1260	1260	502.37	88303.51

we compare the order violations that are detected as potential violations by both tools. For efficiency, we compare the analysis times of Sard and nAdroid.

Table 3.2 compares both in more detail. Its second and third columns show the number of order violations reported by nAdroid and Sard, respectively. For the 16 apps, Sard reports 555 violation pairs while nAdroid reports 1610. Among the violations detected, 350 are reported by both tools and this set of violations contains all the 88 true violations mentioned in nAdroid [26]. Note that each tool reports some UAF order violations missed by the other. We break down the remaining 205 (1260) violations reported by Sard (nAdroid) and manually check them to see whether they are false positives or not in order to understand their precision.

Sard reports fewer false alarms than nAdroid with a good precision. The fifth to the seventh columns of Table 3.2 illustrate the number of true positives and false positives that are reported by Sard alone in the 16 apps. After manual inspection, we found that Sard detects 202 false and 3 true violations, including 2 in `MyTracks_1` and 1 in `MyTracks_2`. All the three true UAFs are missed by nAdroid.

We give an example in `MyTrack_2` in Figure 3.8 to illustrate a typical true UAF case missed by nAdroid. nAdroid fails to detect that the event `run()` executes in a different thread, since nAdroid relies on some filters to remove excessive false alarms heuristically. It has missed the violation pair between line 11 and line 14 due to its aggressive null check filter applied at line 10. However, this check does not guarantee the safety of the field use at line 11.

Meanwhile, we have also checked the 1260 violation pairs detected by nAdroid only, shown in the eighth to tenth columns of Table 3.2. These are found to be all false alarms.

```
1   DataSourceManager dataSourceManager;
2   Handler handler;
3   public void onStart(){
4    dataSourceManager = new DataSourceManager();
5    HandlerThread ht = new HandlerThread();
6    ht.start();
7    handler = new Handler(myThread.getLooper());
8    handler.post(new Runnable(){
9     public void run(){
10     if(dataSourceManager!=null){
11      dataSourceManager.update();}
12    }});}
13   public void onStop(){
14    dataSourceManager = null;}
```

Figure 3.8: A true order violation in `MyTracks_2`.

In total, SARD reports 555 violations with 464 false alarms (and 91 true errors) and NADROID reports 1610 violations with 1522 false alarms (and 88 true errors). We have found 3 more true pairs by issuing three times fewer false positives.

SARD is much more efficient than NADROID, the analysis times of SARD and NADROID for analyzing each app are given in the last two columns of Table 3.2. For the 16 apps used, NADROID takes 88303.51 seconds while SARD is 175 times faster using only 502.37 seconds to finish the analysis. NADROID is slower since it converts all Android's asynchronous events into native Java threads and then applies a heavyweight race detector (e.g., CHORD [56]) to detect potential UAF order violations in a large number of converted threads.

3.5.5 RQ3. SARD vs. NADROID: False Negatives

In this section, we compare the false negative rates of SARD and NADROID in finding UAF order violations of Android apps. We use eight Android apps also

used by nAdroid. These apps were manually injected with 28 UAF order violations detected by a dynamic approach [51], which can be seen as the ground truth. Table 3.3 illustrates the number of violations that are recalled by Sard and nAdroid.

Table 3.3: Comparing Sard and nAdroid in false negatives with regard to the manually injected 28 UAF order violations used in [26]. The numbers in bold indicate that Sard recalls more real violations than nAdroid does.

App Name	#Manually Injected Ordering Violations	#Ordering Violations Detected by nAdroid	#Ordering Violations Detected Sard
Aard	1	1	1
Browser	3	1	**3**
K9 Mail	1	1	1
Mms	6	4	**5**
Music	6	5	5
MyTracks_2	1	1	1
SGT Puzzles	9	8	**9**
Tomdroid	1	0	0
Total	28	21	25

For the eight apps, Sard recalls 25 real UAF order violations in total while nAdroid recalls only 21 violations. For the app `Browser`, Sard finds two more order violations. For `Mms` and `SGT Puzzles`, Sard finds one more violation in each app. In total, we recall 4 more order violations than nAdroid, which shows that Sard has a lower false negative rate (10.7%) than nAdroid (25.0%) in finding UAF order violations for Android apps. We have manually checked the app code to see why Sard outperforms nAdroid. For `Mms`, nAdroid miss 1 more order violation than Sard because the containing method of field access statements is not reachable in its call graph. The other 3 order violations missed by nAdroid in `Browser` and `SGT Puzzles` are due to the fact that nAdroid incorrectly filters them out by its unsound filters ([26], §6.2).

3.5.6 Limitations

Despite the removal of the majority of false positives, the precision of SARD depends on the precision of its underlying pointer analysis used and the precision in reasoning about the feasible control-flow paths between a UAF pair.

Pointer Analysis. SARD leverages the pointer analysis SPARK [38] in SOOT [71]. SPARK can only perform a conservative flow- and context-insensitive may-alias analysis, which can affect the precision of our field usage analysis and the call graph constructed for an app. Some sophisticated pointer analyses [36, 41, 42, 46, 53, 65, 67–69] can be used in future work. In addition, SPARK only partially handles the Java reflection causing incomplete call graph. Integrating reflection analysis [15, 27, 40, 43, 45, 81] is able to improve this problem.

Implicit Control-Flow Path. Another cause of imprecision is the implicit UAF-infeasible paths between two field use statements. Although our light-weight path-sensitive analysis has successfully pruned away a lot of false alarms by analyzing their immediate enclosing branch conditions, there are still situations where SARD is ineffective. For example, a branch condition that represents a null check may involve some complex data-dependencies before a field use statement.

3.6 Chapter Summary

In this chapter, we have presented a new static tool, SARD, for detecting UAF order violations in Android apps. In SARD, we have systematically modeled the asynchronous events in Android and introduced a flow- and context-sensitive analysis to build precise happens-before relations between two events. According to our evaluation, SARD outperforms NADROID by removing its false alarms substantially and discovering its missed true violations with significantly less analysis times.

Chapter 4

Exposing Android Event-Based Races by Selective Branch Instrumentation

4.1 Overview

In the previous chapter, we have introduces our new static tool SARD for detecting UAF order violations in Android. Although SARD improves the state-of-the-art in both precision and efficiency, like other static tools [26, 32], it still has high false positive rates (due to the challenges faced in inferring precisely the pointer/alias information and happens-before relations in Android apps), while dynamic tools [16, 31, 51] exhibit fewer false positives but suffers from code coverage problem.

To reap the best of both worlds, one may apply first static analysis to find all the suspicious event-based races on some *racy statements* in an app and then expose as many true races as possible by performing dynamic analysis.

APEChecker [23] focuses on detecting asynchronous programming errors (including some event-based races as a special case), by first detecting statically suspicious racy statements in an app, then generating program paths from the corresponding event methods to these racy statements and mapping these paths to appropriate events, and finally, dispatching these events during dynamic program execution under an appropriate environment (e.g., with a certain network connection or camera status). During their dynamic analysis, APEChecker tries to expose a suspicious event-based race as a true race by generating appropriate events to reach, i.e., exercise its corresponding racy statements. Unfortunately, racy statements, which are usually guarded by conditionals affected by other specific events, cannot be easily reached this way. In order to satisfy those conditionals during dynamic execution, an exponential number of events must be tried for an app. Therefore, simply generating appropriate events is not able to reach racy statements ideally.

In this chapter, we focus on detecting event-based races in Android apps dynamically. To overcome the challenges faced in generating the right events to satisfy some complex event-dependent conditionals guarding racy statements, we rely on a new selective branch instrumentation technique. Given a suspicious event-based race detected by a static tool, we first collect all the conditionals that can affect the dynamic execution of its racy statements. Then, we analyze systematically each branch to decide whether it should be instrumented or not (i.e., specialized to be always taken or not). We will instrument every selected branch in order to steer the program execution towards the racy statements under consideration. Finally, we execute such an instrumented app dynamically to expose the given suspicious race by generating events appropriately.

The key novelty behind our approach is that we apply a systematic branch analysis to instrument selectively some conditionals guarding a suspicious event-based race. For every such a conditional, analyzing the possible values of the variables used can reveal whether the conditional may be satisfied or not (during some program execution). This allows us to avoid introducing non-existent, i.e., infeasible execution paths caused by instrumenting unsatisfiable conditionals. In addition, we also analyze the safeness of instrumentation on every conditional, reducing unexpected crashes during dynamic execution. For some conditionals that are not suitable for instrumentation (due to crashes), our branch analysis will identify the definition statements for all the variables used and dispatch appropriate events to reach these definition statements so that these conditionals can be evaluated dynamically.

We have implemented our approach for detecting event-based races in a tool, called Sieve (**S**elective **I**nstrumentation for **Eve**nt-based races). As the closely related tool APEChecker [23], is not open-sourced, we have evaluated Sieve against a baseline tool (implemented by ourselves) that simply generates events to trigger the event methods in a suspicious event-based race by mimicking how APEChecker works. Our set of benchmarks consists of 25 real-world Android apps containing 190 suspicious event-based races reported by Sard, a static tool for detecting event-based races in Android apps [74], of which 18 are true races. Among all the suspicious races, Sieve has successfully reached 90 races (47.3%) during the dynamic execution and exposed 16 true races without any false alarm, while the baseline tool has reached only 56 (29.4%) races and found only 14 true races.

In summary, this chapter makes the following contributions:

- We present a novel selective branch instrumentation technique that can expose event-based races more effectively than the state of the art, with a low false positive rate.

- We introduce an effective race detection tool, Sieve, for finding event-based races in Android apps dynamically, based on selective branch instrumentation.

- We evaluate Sieve with 25 real-world Android apps, by demonstrating that our approach is more effective than the state of the art in detecting event-based races.

The rest of the chapter is organized as follows: Section 4.2 discusses the related work. Section 4.3 motivates our approach. We introduce Sieve in Section 4.4 and evaluate it in Section 4.5. Finally, Section 4.6 concludes the chapter.

4.2 Related Work

Event-Based Race Detection We have already discussed the previous event-based race detection tools in Section 3.2. Currently, dynamic tools [16, 31, 51] suffer from the classic code coverage problem and the static tools [26, 32, 44, 62, 74] are susceptible to false positives due to, for example, imprecision in the alias information and happens-before relations used.

An effective approach for reducing false positives reported by static tools is to further expose the race warnings from static tools dynamically. Both ERVA [33] and RacerDroid [70] can take the results from existing tools and replay the corresponding events to invoke callbacks reaching the racy statements in a correct order. However, both tools still suffer from the code coverage problem, as ERVA

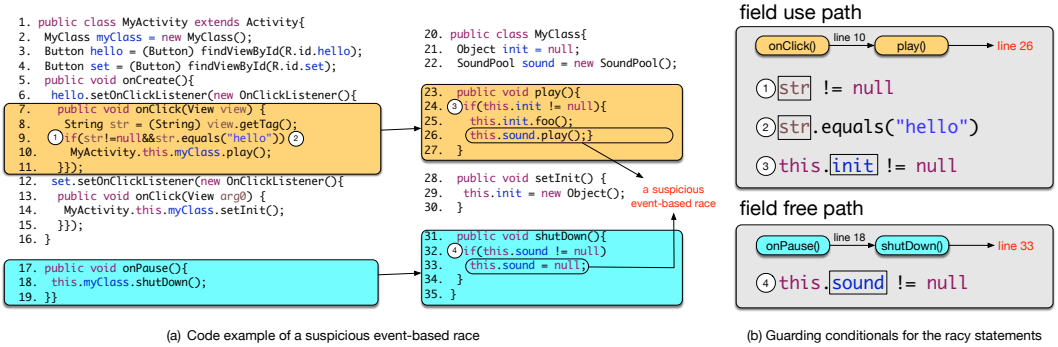
can only work together with a dynamic tools (due to its design on building its event dependency graphs dynamically) and RacerDroid relies on dynamic exploration to obtain the callbacks reaching racy statements. The state-of-the-art APEChecker [23] exposes event-based races by finding and rescheduling statically the events that can invoke the callbacks reaching a pair of racy statements. However, racy statements are usually guarded by event-dependent conditionals, limiting its effectiveness (as evaluated in the chapter).

GUI Testing. Except for the tools designated for race detection, general-purpose GUI testing tools [8, 19, 28, 48, 49, 52, 54, 55, 61, 66] can also be leveraged to exposed event-based races. These tools sends a stream of randomized or orchestrated user events into system acting as a stress test on applications, which is able to trigger concurrency related exceptions (e.g., null pointer exception) with enough events sent. However, general-purpose GUI testing tools are poor in test coverage resulting in worse performance in both effectiveness and efficiency than tools designed for detecting concurrency bugs.

4.3 A Motivating Example

Figure 4.1 depicts our motivating example. Figure 4.1(a) is abstracted from two real apps, PhoneFoneFun [12] and VitoshaDM [13], with a few modifications for illustration purposes. Existing static tools for detecting event-based races [26, 32, 74] may report a suspicious race between line 26 and line 33 on the field `sound` of a `MyClass` object pointing to a `SoundPool` object. However, due to their high false positive rates, static tools can only flag this as a race warning.

By inspecting the code manually, we found that this race warning is indeed a true race, since there exists two dynamic execution paths from its two correspond-



(a) Code example of a suspicious event-based race

(b) Guarding conditionals for the racy statements

Figure 4.1: A motivating example of Sieve.

ing event methods to the racy statements at lines 26 and 33, as shown in Figure 4.1(b), so that all their guarding conditionals can be satisfied. When the user clicks the `hello` button on the screen (line 3), its associated callback `onClick()` (lines 7-11) will be invoked. Under certain circumstances, `onClick()` will invoke `play()`, reaching the field use statement at line 26. On the other hand, an incoming call can trigger `onPause()`, which will invoke `shutDown()`, reaching the field free statement at line 33. Since the click event and incoming call event are generated asynchronously, `onPause()` can be possibly executed before `onClick()`. In this case, the field `sound` pointing to a `SoundPool` object would be first freed and then used later, causing an event-based race, and subsequently, a null pointer exception being thrown.

4.3.1 Existing Techniques

Existing dynamic tool APEChecker [23] will usually fail to expose the race warning for the two racy statements at lines 26 and 33 as a true race. The main limitation is that it cannot systematically generate the events required to satisfy all the guarding conditionals in Figure 4.1(b) so that the two racy statements can be exercised.

APEChecker [23] will first perform static analysis to construct the two paths from `onClick()` and `onPause()` to the two racy statements at lines 26 and 33, as shown in Figure 4.1(b). APEChecker will then generate and execute the corresponding events so that `onClick()` and `onPause()` are executed. Targeting the conditionals, APEChecker tries to alleviate this problem by supporting lightweight modeling of system setting (e.g., network connection, camera condition) or user inputs (e.g., email, phone number). For example, if `WifiManager.isWifiEnabled()` appears in a relevant conditional, APEChecker will set the WiFi to a specific status before the app executes. However, these strategies could only affect a small part of the conditionals. In this example, APEChecker will fail to reach line 26, as its guarding conditional `this.init != null` will always evaluate to false, unless a click event is generated for the `set` button (to initialize the `init` field of a `MyClass` object).

4.3.2 Sieve

To expose event-based races more effectively than the prior art, Sieve will selectively instrument, i.e., specialize some guarding conditionals for racy statements (as being either always taken or always not taken) so that more racy statements can be exercised. To reduce false positives and unexpected crashes, Sieve selects only certain conditionals for instrumentation based on a systematic branch analysis. For our example, Sieve can successfully reach the racy statements at lines 26 and 33 and expose a true race in between.

Given the suspicious race at lines 26 and 33, Sieve first collects their guarding conditionals listed in Figure 4.1(b) for the use path to line 26 and the free path to line 33. Then Sieve performs a systematic branch analysis on each conditional to determine whether it should be instrumented or not. To avoid introducing

infeasible execution paths, we need to check whether a conditional can be satisfied along its path by analyzing the possible values of all the variables appearing in the conditional. In our example, the variables that need to be analyzed are boxed in Figure 4.1(b).

For the two conditionals, ① and ②, the variable `str` used therein is defined as a return value of `getTag()` of a string type. We can reasonably assume that `str` may be initialized with "`hello`" so that both ① and ② can be satisfied during program execution. As for the conditional ④, `this.sound` used therein is also used in the racy statement at line 33. In that case, ④ is left unchanged in order to ensure the correct execution of the racy statement at line 33.

Determining whether the conditional ③ at line 24 can be satisfied or not during program execution is more complex, since we need to reason about the possible values taken by the field `init` of a `MyClass` object. Sieve first finds all the definition statements of `init` to see if `init` may be assigned with a value that can make ③ satisfied. In the code snippet, `init` is initialized with a new object at line 29 that meets this requirement. Before drawing the conclusion, Sieve will also need to inspect whether the statement at line 29 can be executed before ③. Indeed, this statement can be reached from the callback `onClick()` (lines 13-15) associated with the `set` button (line 4). Due to the lack of any happens-before relation between the two `onClick()` methods, the `onClick()` (lines 13-15) can be possibly executed before the `onClick()` (lines 7-11). However, if the latter always happens-before the former instead, then ③ can never be satisfied, making the reported race a false positive.

After having checked the satisfiability of all the four conditionals, Sieve will proceed to check the safeness for selective instrumentation. Regarding ③ at line 24 as always satisfiable is not safe since the statement at line 25 is data-dependent on it.

Without this check, a null pointer exception may be thrown at line 25. Therefore, instead of instrumenting ③, SIEVE will opt to handle it during program execution as discussed below. The other two conditionals, ① and ②, will be specialized to true so that line 10 is always executed. As for ④, it will not be instrumented as discussed earlier.

SIEVE now executes the instrumented app to see if the given suspicious race can be exposed as a true race. For a `MyActivity` object, SIEVE generates an incoming call event to cause `onPause()` to be invoked, so that the field free statement at line 33 can be reached. To reach the field use statement at line 26, line 10 will be reached initially since ① and ② are instrumented to be always taken. To satisfy ③, SIEVE will first click the `set` button to invoke `onClick()` (lines 13-15) so that the `init` field is initialized at line 29. SIEVE will then click the `hello` button to invoke `onClick()` (lines 7-11), so that the statement at line 26 is reached.

Finally, SIEVE reports the suspicious race on the field `sound` of a `MyClass` object as a dangerous race, since a null pointer exception will be thrown if line 33 happens to be executed before line 26.

4.4 Approach

We first give an overview of SIEVE (Section 4.4.1). We then describe its different analysis phases in more detail (Sections 4.4.2 to 4.4.4). Finally, we discuss its dynamic execution phase (Section 4.4.5).

4.4.1 Overview

Figure 4.2 gives the workflow of SIEVE. Given an Android apk file, we first apply the static tool, SARD [74], to detect suspicious event-based races in the app. Each

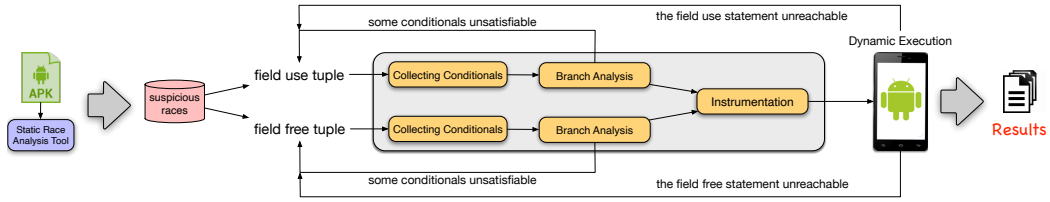


Figure 4.2: Workflow of Sieve.

race warning consists of (1) a pair of racy statements s_u and s_f that perform the use and free operations on a shared object, respectively; (2) a pair of event (callback) methods e_u and e_f, which can react to user or system events and reach the racy statements s_u and s_f, respectively; and (3) a pair of context sets $\mathbb{C}_u$ and $\mathbb{C}_f$. A context $ctx = [I_1, I_2, \cdots, I_n]$ is essentially a list of invocation statements leading from an event method to the method where the corresponding racy statement resides in.

For a given race, SIEVE operates on a pair of triples, $\langle \tau_u, \tau_f \rangle$, such that $\tau_p = \langle s_p, e_p, ctx_p \rangle$, where $p \in \{u, f\}$. Here, s_p is a racy statement, e_p is the corresponding event method leading to s_p, and $ctx_p \in \mathbb{C}_p$.

For every such a triple $\langle s_p, e_p, ctx_p \rangle$, SIEVE first collects all the conditionals that may prevent the execution from reaching s_p from e_p under context ctx_p. SIEVE then applies a systematic branch analysis to each conditional to check whether it can be satisfied or not during some program execution. If there exists a conditional that can never be satisfied, SIEVE will pick a different context ctx'_p from $\mathbb{C}_p$ and start over again. If all the guarding conditionals for τ_u and τ_f can be satisfied under some contexts ctx_u and ctx_f, respectively, SIEVE will proceed to check the safeness for instrumenting each conditional. Then, SIEVE will selectively instrument certain branches by replacing each branch with true or false appropriately to steer the execution towards the racy statement guarded.

Finally, SIEVE will execute the instrumented app by generating the events as desired in order to expose the given suspicious race as a true race by observing any null pointer exception being thrown. If no race is found, SIEVE will try different contexts $\mathbb{C}_u$ and $\mathbb{C}_f$ until a race has been exposed or a pre-set time budget has been exhausted.

$$\frac{\tau : \langle s, e, ctx \rangle \quad t \in \{s\} \cup ctx}{\{t\} \subseteq \mathcal{W}} \quad \text{[C-INITIALIZE]}$$

$$\frac{\begin{array}{c} w \in \mathcal{W} \quad t \in stmt(\mathcal{M}_w) \\ t \text{ is a conditional statement} \quad t \xrightarrow{g} w \end{array}}{\{t\} \subseteq \mathcal{W} \quad \{t\} \subseteq \mathcal{W}} \quad \text{[C-COLLECT]}$$

Figure 4.3: Rules for collecting conditionals.

4.4.2 Collecting Conditionals

For a triple $\langle s, e, ctx \rangle$, SIEVE collects all the guarding conditionals that can affect the reaching racy statement s by applying the rules given in Figure 4.3. By [C-INITIALIZE], we initialize the *worklist* $\mathcal{W}$ with the racy statement s and all the invocation statements in the list of context ctx. Afterwards, we apply [C-COLLECT] iteratively. For a conditional statement t that is in the same method $\mathcal{M}_w$ that contains a statement w in $\mathcal{W}$, if w is control dependent on t, we consider t is a guarding conditional of w denoted as $t \xrightarrow{g} w$. Then we include t to worklist $\mathcal{W}$. After all the statements in the worklist $\mathcal{W}$ have been processed by applying [C-

COLLECT], we can extract all the conditional statements from $\mathcal{W}$ to form a set of conditionals, $\mathcal{C}$, by applying [C-FINALIZE].

For a triple $\langle s, e, ctx \rangle$, SIEVE collects all the guarding conditionals $\mathcal{C}$ that can affect the reaching racy statement s by applying the two rules given in Figure 4.3. By [C-INITIALIZE], we initialize the *worklist* $\mathcal{W}$ with the racy statement s and all the invocation statements in the list of context ctx. Afterwards, we apply [C-COLLECT] iteratively. For a conditional statement t that is in the same method $\mathcal{M}_w$ that contains a statement w in $\mathcal{W}$, if w is control dependent on t, we consider t as a guarding conditional of w denoted as $t \xrightarrow{g} w$. Then we include t to the worklist $\mathcal{W}$ and the conditional set $\mathcal{C}$.

4.4.3 Branch Analysis

To support our selective branch instrumentation, SIEVE performs a systematic branch analysis for all the collected conditionals in $\mathcal{C}$ to check whether they can be satisfied during program execution under their designated contexts. There are two steps, definition tracing and value range analysis, which are described below.

4.4.3.1 Definition Tracing

For each conditional $c \in \mathcal{C}$, SIEVE finds the definition statements of the variables used in c by performing an inter-procedural backward tracing context-sensitively by applying the rules given in Figure 4.4. Let us consider [T-DEFINITION] first. For every variable v used in c, if d is a definition statement indicated by $d \in DefStmt(v)$, then $\langle v, d \rangle$ is recorded in $\mathcal{D}$. For each $\langle v, d \rangle \in \mathcal{D}$, the remaining three rules are applied iteratively.

For a definition statement $d : x = l$ of a variable v, if l is a local variable, then [T-LOCAL] will be applied (iteratively). This rule removes the original definition

$\langle v, d\rangle$ from $\mathcal{D}$ and adds a set of new ones, $\{\langle v, d'\rangle \mid d' \in DefStmt(l)\}$, to $\mathcal{D}$, where d' is one of the definition statements of l.

[T-PARAMETER] is applied when p_i in the definition statement $d : x = p_i$ is a formal parameter of its containing method $\mathcal{M}_d$. Based on p_i and the invocation statement I under the given context ctx, we can obtain the corresponding actual argument a_i, which is used to find its definition statement d'.

[T-FIELD] is applied when the underlying definition statement is a load $d : x = _.f$, where f can be either a static field or an instance field. First of all, f cannot be a field of any Android framework class since it would be otherwise hard to know where its definitions are. In addition, f cannot be the same field accessed by the racy statement s, since instrumenting any conditional involving $_.f$ will not be safe (as demonstrated with ④ in our motivating example). The function $fieldDefStmt()$ returns the set of definition statements for the field $_.f$. In the case of an instance field $_.f$, $x.f = \cdots$ can be potentially a definition statement if x may be aliased with the base pointer $_$. Given a field definition statement d', the function *entries* returns the set of the callback methods reaching d'. To ensure that the definition statement d' can be executed before the conditional c where v is used, e must not happen-before $e_{d'}$, denoted by $e \not\prec e_{d'}$.

4.4.3.2 Value Range Analysis

Let $\mathcal{V} \cup \{\mathbf{0}\}$ be the set of values for all the variables, where $\mathbf{0} \notin \mathcal{V}$ is a special value. Conditionals containing variables that may have the special value $\mathbf{0}$ will not be instrumented later. In this value range analysis phase, SIEVE will analyze the possible values in $\mathbb{V} \in 2^{\mathcal{V}\cup\{\mathbf{0}\}}$ taken by each variable that appears in a conditional, by applying the rules given in Figure 4.5.

$$\frac{c \in \mathcal{C} \quad v \in variables(c)}{\mathcal{D} = \mathcal{D} \cup \{\langle v, d\rangle \mid d \in DefStmt(v)\}} \quad \text{[T-DEFINITION]}$$

$$\frac{\langle v, d\rangle \in \mathcal{D} \quad d : x = l \quad l \text{ is a local variable}}{\mathcal{D} = \mathcal{D} \setminus \{\langle v, d\rangle\} \cup \{\langle v, d'\rangle \mid d' \in DefStmt(l)\}} \quad \text{[T-LOCAL]}$$

$$\frac{\begin{array}{c}\tau : \langle s, e, ctx\rangle \quad \langle v, d\rangle \in \mathcal{D} \quad I \in ctx \quad I \text{ invokes } \mathcal{M}_d \\ p_i \text{ is } \mathcal{M}_d\text{'s formal parameter} \\ d : x = p_i \quad a_i = argument(p_i, I)\end{array}}{\mathcal{D} = \mathcal{D} \setminus \{\langle v, d\rangle\} \cup \{\langle v, d'\rangle \mid d' \in DefStmt(a_i)\}} \quad \text{[T-PARAMETER]}$$

$$\frac{\begin{array}{c}\tau : \langle s, e, ctx\rangle \quad \langle v, d\rangle \in \mathcal{D} \quad d : x = _.f \\ _.f \text{ is not a field of an Android framework class} \\ _.f \text{ is not the same field accessed by } s \\ \mathcal{F} = \{d' \mid d' \in fieldDefStmt(_.f) \quad e_{d'} \in entries(d') \quad e \not\prec e_{d'}\}\end{array}}{\mathcal{D} = \mathcal{D} \setminus \{\langle v, d\rangle\} \cup \{\langle v, d'\rangle \mid d' \in \mathcal{F}\}} \quad \text{[T-FIELD]}$$

Figure 4.4: Rules for definition tracing.

The rule [V-PARAMETER] is applied if the variable v is defined in terms of a parameter of a callback method. Since the values of a callback parameter can vary widely depending on system or user actions, we assume reasonably that v can take any value from $\mathcal{V}$. For similar reasons, we have also made the same assumption as when v is defined in terms of a field of an Android framework class ([V-FRAMEWORK]). As for the situation when v is defined with a return value of a method ([V-METHOD]), in most of the cases, the possible return values of this method can satisfy the conditional containing v, even if we apply heavyweight context- and path-sensitive analysis. Therefore, we apply the same assumption in this rule.

The rules [V-VALUE-NEW] and [V-VALUE-CONST] for constant initializations in both cases are self-explanatory.

$$\frac{\langle v, d\rangle \in \mathcal{D} \quad d : x = p_i \quad p_i \text{ is a formal parameter of a callback method}}{\mathcal{V} \subseteq \mathbb{V}_v} \quad \text{[V-PARAMETER]}$$

$$\frac{\langle v, d\rangle \in \mathcal{D} \quad d : x = foo(_)}{\mathcal{V} \subseteq \mathbb{V}_v} \quad \text{[V-METHOD]}$$

$$\frac{\langle v, d\rangle \in \mathcal{D} \quad d : x = _.f \quad _.f \text{ is a field of an Android framework class}}{\mathcal{V} \subseteq \mathbb{V}_v} \quad \text{[V-FRAMEWORK]}$$

$$\frac{\langle v, d\rangle \in \mathcal{D} \quad d : x = \textbf{new } T()}{\{\textbf{new } T()\} \subseteq \mathbb{V}_v} \quad \text{[V-VALUE-NEW]}$$

$$\frac{\langle v, d\rangle \in \mathcal{D} \quad d : x = constant}{\{constant\} \subseteq \mathbb{V}_v} \quad \text{[V-VALUE-CONST]}$$

$$\frac{\tau : \langle s, e, ctx\rangle \quad \langle v, d\rangle \in \mathcal{D} \quad d : x = _.f \quad _.f \text{ is possibly the same field accessed by } s}{\{\mathbf{0}\} \subseteq \mathbb{V}_v} \quad \text{[V-SAME]}$$

$$\frac{\langle v, d\rangle \in \mathcal{D} \quad d : x = b \in otherForm(a)}{\{\mathbf{0}\} \subseteq \mathbb{V}_v} \quad \text{[V-RESERVE]}$$

Figure 4.5: Rules for value range analysis.

The rule [V-SAME] handles the case when v is defined in terms of a field $_.f$, where the base pointer $_$ is aliased with the base pointer of the field accessed by the racy statement s (as demonstrated by ④ in our motivating example). To avoid crashing the app, any conditional involving this field access will not be instrumented.

The rule [V-RESERVE] is applied if the variable v is defined in a way that is not captured by the other rules. We have observed that v is frequently defined locally

in this case. Therefore, we assume that its possible values can be obtained during program execution. No instrumentation for v is actually needed.

4.4.4 Selective Branch Instrumentation

For the set of conditionals $\mathcal{C}$, collected for $\tau : \langle s, e, ctx \rangle$ (Section 4.4.2), SIEVE will select appropriate conditionals to instrument based on the value ranges collected for the variables appearing in these conditionals (Section 4.4.3), by applying the rules given in Figure 4.6.

We instrument a conditional only if it can be satisfied during some program execution. Otherwise, a race exposed along some infeasible program paths represents a false positive only.

The two rules, [S-CONST] and [S-VARIABLE], are used to check the satisfiability of a conditional c. Here, $\trianglerighteq$ denotes an abstract relational operator that represents a concrete relational operator such as $>, \neq$ and $\leq$. Given two possible value sets, $\mathbb{V}_1$ and $\mathbb{V}_2$ (with neither containing $\mathbf{0}$), $satisfy(\mathbb{V}_1, \mathbb{V}_2, \trianglerighteq)$ returns true if and only if v_1 and v_2, where $v_1 \times v_2 \in \mathbb{V}_1 \times \mathbb{V}_2$, are type-compatible and their possible values are satisfied with respect to $\trianglerighteq$. As mentioned above, if the set of possible values of any variable that appears in c includes $\mathbf{0}$, then c will not be instrumented. If a conditional c can be satisfied during program execution, it will be recorded in $\mathcal{S}$.

The rule [S-SAFENESS] ensures the safeness for instrumenting a conditional. Replacing a conditional by a fixed true/false outcome may lead to unexpected crashes or infinite loops. In addition, a guarding conditional c for $\tau : \langle s, e, ctx \rangle$ can be reached from not only e but also some other callback methods. Instrumenting c may falsely introduce some execution paths of other callbacks that should otherwise not be executed, causing unexpected crashes. Therefore, before performing

$$\frac{c : v \trianglerighteq constant \in \mathcal{C} \quad \mathbf{0} \notin \mathbb{V}_v \quad satisfy(\mathbb{V}_v, \{constant\}, \trianglerighteq)}{\{c\} \subseteq \mathcal{S}} \quad \text{[S-CONST]}$$

$$\frac{c : v_i \trianglerighteq v_j \in \mathcal{C} \quad \mathbf{0} \notin \mathbb{V}_{v_i} \quad \mathbf{0} \notin \mathbb{V}_{v_j} \quad satisfy(\mathbb{V}_{v_i}, \mathbb{V}_{v_j}, \trianglerighteq)}{\{c\} \subseteq \mathcal{S}} \quad \text{[S-VARIABLE]}$$

$$\frac{\tau : \langle s, e, ctx \rangle \quad c_s \in \mathcal{S} \quad e_o \in entries(c_s) \quad i \in stmt(\mathcal{M}_{c_s}) \quad c_s \xrightarrow{g} i \quad c_s \overset{u}{\not\rightarrow} i \quad e_o \neq e \quad e_o \not\prec e}{\{c\} \subseteq \mathcal{I}} \quad \text{[S-SAFENESS]}$$

Figure 4.6: Rules for selective branch instrumentation.

instrumentation on a satisfiable conditional $c_s \in \mathcal{S}$, SIEVE also checks the safeness for doing so using this rule.

Let us now examine [S-SAFENESS] in more detail. For a statement i, whose guarding conditional is c_s, denoted as $c_s \xrightarrow{g} i$, i should not be use-dependent on c_s, denoted $c \overset{u}{\not\rightarrow} i$ (to reduce program crashes or infinite loops introduced by instrumenting c_s). In this chapter, i is considered to be *use-dependent* on c_s if it uses a variable in c_s without any re-definition for the used variable in between. In addition, for every other callback e_o that can reach c_s, e_o should not happen-before e, denoted $e_o \not\prec e$ (for the purposes of reducing crashes and infinite loops introduced again).

It is important to point out that it is impossible to avoid introducing crashes or infinite loops unless we can reason about the precise intention that programmers have in mind in writing their code. The objective of [S-SAFENESS] is to reduce their occurrences in order to maximize the effectiveness of SIEVE. As evaluated

in Section 4.5, SIEVE induces a few crashes and no infinite loops for the 25 apps evaluated. Interestingly, an infinite loop introduced may still allow a race to be exposed due to the null pointer exception thrown inside the loop. Due to the potential existence of an infinite loop, a time budget is set when exercising an event (Section 4.4.5).

By applying the rules in Figure 4.6, we will obtain the set of conditionals to be instrumented in $\mathcal{I}$. Based on the branch guarding a racy statement, we instrument its outcome as true or false whichever is appropriate to ensure that the racy statement can be reached during the instrumented program execution. For the conditionals that are satisfiable but unsafe for instrumentation, we record the callback methods that make these conditionals satisfied. Those callbacks are referred to as *settable callbacks* and will be activated by generating appropriate events during dynamic execution (as described below).

Finally, there is a caveat on a semaphore mechanism used in detecting event-based races dynamically. In order to expose the race between τ_u and τ_f, we must ensure that s_f is executed before s_u. If both statements are executed in the same thread, simply invoking e_f before e_u will ensure this correct execution order as desired. However, if both statements are executed in two different threads, the desired execution order for s_f and s_u cannot be guaranteed this way. Instead, we will proceed similarly as in APECHECKER [23] by using a semaphore mechanism. SIEVE inserts a P operation before the use statement s_u and a V operation after the free statement s_f. As a result, we will not move past s_u until s_f has been executed. The waiting operation here simulates a time-consuming task, without changing the behaviors of the app.

Figure 4.7 illustrates an inter-thread event-based race. In the original code given in Figure 4.7(a), the statements at lines 3 and 7 are supposed to run in two different

```
Thread 1:
1. public void use(){
2.  if(field!=null){
3.   field.foo();
4.  }
5. }

Thread 2:
6. public void free(){
7.  field = null;
8. }
```

(a) original code

```
1. public void use(){
2.  synchronized(signal){
3.   if(field!=null){
4.    signal.wait();
5.    field.foo();}}
6. }

7. public void free(){
8.  synchronized(signal){
9.   field = null;
10.  signal.notify();}
11.}
```

(b) instrumented code

Figure 4.7: An inter-thread event-based race.

threads. In order to expose the race, the correct execution order should be line 2 → line 7 → line 3. It is not easy to achieve this order by executing sequentially `free()` and `use()` in that order. By inserting the semaphore operations as shown in Figure 4.7(b), `use()` should be executed before `free()` in order to maximize the chances for exposing the race at run time.

4.4.5 Dynamic Execution

SIEVE tries to expose the event-based race (if any) between s_u and s_f by executing dynamically the instrumented app on an Android emulator. It maps the callback methods to corresponding user or system actions and simulates these actions by `Android Debugging Bridge(adb)` [1]. Currently, SIEVE focuses on modeling two main types of callback methods: (1) callback methods that react to user actions on UI widgets (e.g., `onClick()`) and (2) lifecycle callbacks of components (e.g., `onPause()`). In addition, SIEVE also inspects the information from `Android Logcat` [9] to monitor any exception raised due to an exposed race. SIEVE reports

an event-based race if a null pointer exception is thrown when s_f is executed before s_u.

For each tuple $\tau_p : \langle s_p, e_p, ctx_p \rangle$, where $p \in \{f, u\}$, SIEVE generates a sequence of events $\mathcal{Q}_p$ consisting of some *navigation events*, some *settable events* and a *race event*. Navigation events are used to steer the program to the *target components* that contains e_p. SIEVE finds the callbacks that can reach the statements starting target components (e.g. `startActivity()`) and then generates the corresponding navigation events. After having reached the target component, SIEVE will send the settable events to invoke the settable callbacks described in Section 4.4.4 and then the race event to invoke e_p. For each event in $\mathcal{Q}_p$, SIEVE currently allows their corresponding callbacks to execute for up to 2 seconds (to avoid getting stuck in an infinite loop that may be introduced, as discussed earlier in Section 4.4.4).

To expose the race between τ_u and τ_f, the execution order of $\mathcal{Q}_u$ and $\mathcal{Q}_f$ is determined as follows. If there is no semaphore injected during the instrumentation, $\mathcal{Q}_f$ will be executed first. Otherwise, $\mathcal{Q}_u$ will be executed first, as illustrated in Figure 4.7.

4.5 Evaluation

Our evaluation aims to show that SIEVE can advance the state-of-the-art dynamic techniques for finding event-based races in Android apps. We address three research questions:

- **RQ1.** Can SIEVE be used as a practical tool for exposing event-based races dynamically in Android apps?

- **RQ2.** Is SIEVE more effective than existing dynamic tools?

- **RQ3.** Can SIEVE generate fewer crashes and fewer false positives by instrumenting branches selectively than fully?

4.5.1 Implementation

We have implemented SIEVE in Java (in 8K LOC) on top of several open-source tools. We rely on FLOWDROID [14] to decompile an Android apk file and construct its call graph. For a given app, SIEVE operates on its Jimple intermediate representation (IR) supported by Soot [71] for performing its instrumentation and repackaging the instrumented IR into an apk file. During the execution phase, SIEVE sends commands through Android Debugging Bridge (adb) [1] to generate events (e.g., clicks and incoming calls), which can invoke the corresponding callbacks. For some of the callbacks that are triggered by interacting with some UI widgets on the screen, SIEVE utilizes Gator [77] to map the callbacks to the ids of the UI widgets. This allows us to select the correct UI widgets to perform appropriate operations (e.g., clicks) as desired. Finally, we make use of Logcat [9] to monitor the exceptions thrown during the execution in order to expose the exception-causing races in an Android app.

4.5.2 Methodology

We have selected a set of 25 real-world Android apps from F-Droid [6], a repository of open-source Android apps. These apps are obtained based on two selection criteria: (1) no user credentials are required, since the human efforts will otherwise be needed during the dynamic execution; and (2) no unsupported events are required to reach the target components or racy statements, since SIEVE currently supports for mapping parts of the callback methods to user or system events only. There are 190 suspicious races reported in the 25 apps by SARD [74]. Of the 190 race

warnings reported, 18 are true races and 172 are false positives, found by manual code inspection.

We address RQ1 – RQ3 as follows. To evaluate RQ1, we analyze Sieve's recall and false positive rates, together with its analysis times. To evaluate RQ2, we compare Sieve with APEChecker, a state-of-the-art dynamic tool [23], which does not perform any branch instrumentation. However, as APEChecker is not open-sourced, we will evaluate Sieve against a version of Sieve, named Sieve-ZB, that simply generates events to trigger the event methods in a suspicious event-based race with the aforementioned semaphore mechanism by mimicking how APEChecker works. By inspecting manually the 190 suspicious races in the 25 apps, we found that APEChecker does not perform any instrumentation on any of their guarding conditionals as these conditionals cannot be handled by its lightweight model (Section 4.3.1). Thus, Sieve-ZB is expected to behave similarly as APEChecker for the 25 apps considered. To evaluate RQ3, we compare Sieve with a version, named Sieve-FB, that instruments all the guarding conditionals for a given pair of racy statements.

Our evaluation is conducted on a quad-core i5-6500 3.2GHz machine with 16GB RAM running Ubuntu 16.04 LTS. Sieve executes an app on a Nexus 4 emulator with API level 28. The analysis time for each app is the average of three runs.

4.5.3 Results and Analysis

Table 4.1 presents our results, divided into five parts separated by "||". Part 1 gives the information about each app, including its name, its LOC and the number of suspicious races reported statically. In Column 3, the number x inside (x) for an app is the number of true races in the app. Parts 2 – 4 contain the information for

Table 4.1: Comparing Sieve (selective branch instrumentation) with Sieve-ZB (no branch instrumentation) and Sieve-FB (full/all branch instrumentation) in analyzing 25 Android apps containing 190 race warnings (including 18 true races).

App	LOC	#Suspicious Races Found Statically (True Races)	RQ1 Sieve: Selective Instrumentation				RQ2 Sieve-ZB: Non-Instrumentation			RQ3 Sieve-FB: Full-Instrumentation			Time for Sieve (secs)
			#Infeasible Races	#Unreached Races	#Crashes	#Reached Races (Reported Races)	#Unreached Races	#Crashes	#Reached Races (Reported Races)	#Unreached Races	#Crashes	#Reached Races (Reported Races)	
AsciiCam	2808	2	0	1	0	1	1	0	1	1	1	1(1)	66.2
CMIS Browser	12356	1	0	1	0	0	1	0	0	1	0	0	58.7
Cool Mic	3083	11	0	11	0	0	11	0	0	11	2	0	423.0
Duorem	3728	2	0	0	0	2	2	0	0	0	0	2	98.9
GTFSOffline	4659	2	0	2	0	0	2	0	0	1	1	1(1)	78.1
Ithaka Game	2187	1	0	0	0	1	1	0	0	1	1	0	41.1
KIBA	2069	1(1)	0	0	0	1(1)	0	0	1(1)	1	1	0	39.5
KitchenTimer	1236	5	0	5	0	0	5	0	0	5	2	0	218.3
Mitzuli	9058	13	2	11	0	0	13	0	0	13	10	0	958.5
MouseApp	5856	4	0	0	0	4	0	0	4	0	0	4	150.8
Nounours	5814	2	0	2	0	0	2	0	0	2	1	0	118.5
Nusic	8520	6	0	0	0	6	0	0	6	0	0	6	434.2
Patolli	1886	13	0	13	2	0	13	0	0	13	7	0	419.1
PhoneFoneFun	623	4	0	1	0	3	2	0	2	1	0	3	215.3
Rick App	1010	6	0	5	0	1	6	0	0	5	0	1	232.2
Ringdroid	7819	26	0	6	2	20	25	0	1	5	5	18(5)	1202.7
SicMu Player	5568	18(2)	15	3	0	0	18	0	0	11	8	7(7)	382.1
SliderSynth	3015	5	0	4	0	1	4	0	1	5	1	0	521.6
Speech Trainer	1851	12	0	6	0	6	6	0	6	12	0	0	984.3
Tallyphant	1303	22	2	0	0	20	8	0	14	2	2	20	892.2
Vector Pinball	9094	23(13)	0	8	3	15(13)	8	0	15(13)	8	6	15(13)	1237.7
Vitosha DM	390	2(2)	0	0	0	2(2)	2	0	0	0	0	2(2)	62.7
Weechat	9264	5	0	1	0	4	3	0	2	1	0	4	318.2
YucataEnvoy	3641	1	0	1	0	0	1	0	0	1	0	0	35.6
Zxing	29204	3	0	0	0	3	0	0	3	1	1	2	246.1
Total	136042	190(18)	19	81	7	90(16)	134	0	56(14)	104	49	86(29)	9435.6

RQ1 – RQ3, respectively. Part 5 gives the time taken by SIEVE for analyzing each app.

4.5.3.1 RQ1. Practicality

SIEVE has succeeded in exposing 16 out of 18 true races in the 25 apps (achieving a recall rate of 88.9%) without producing any false positive. The analysis time for each app includes the time of detecting suspicious races using SARD and the running time of SIEVE its own. It takes a total of 9435.6 seconds (about 2.6 hours) for analyzing all the 25 apps, with an average of 377.4 seconds (6.3 minutes) per app. As for the previous state-of-the-art APECHECKER, it spends 6.7 minutes on each app, which is slightly longer than SIEVE. Below we analyze the results given in Column "RQ1" to demonstrate the practicality of SIEVE.

In Column "#Infeasible Races" for RQ1, we give the number of races that are doomed to be infeasible for each app by our branch analysis. A suspicious race is *infeasible* if one of its guarding conditionals can never be satisfied under any given context ([S-VARIABLE] and [S-CONST]). For infeasible races, no further dynamic analysis is necessary. In total, 19 infeasible races are found.

In Column "#Unreached Races" and Column "#Crahses" for RQ1, we give the number of unreached races and crashes. We have manually inspected the source code of the 17 apps containing unreached races (totaling 81). The reasons behind and their percentage contributions are depicted in Figure 4.8.

- **Uninstrumented Conditionals.** SIEVE instruments selectively some guarding conditionals for a suspicious race conservatively in order to reduce crashes and false positives. A total of 23 unreached races fall into this category, since some of their guarding conditionals (not instrumented by our rules in Figure 4.6) are not satisfied dynamically.

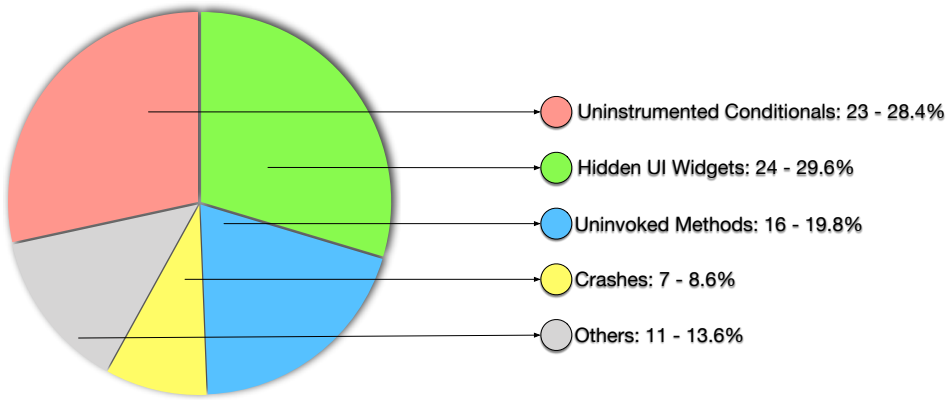


Figure 4.8: Different causes for unreached races.

- **Hidden UI Widgets.** For an event method reaching the racy statement of a suspicious race, its corresponding UI widget can only be shown under certain conditions. Currently, SIEVE models this only partially (e.g., a button in a popup-dialog), resulting in 24 suspicious races not to be reached. Within this category, we have missed 2 true races in `SicMu Player`, which are the only two missed in the 25 apps.

- **Uninvoked Methods.** Some of system methods can only be triggered by the Android system. Specifically, if the statement `unbindService()` is called within callback methods, `onServiceDisconnected()` can be invoked by the system under the condition that the process hosting the service has crashed or has been killed by the Android system. Currently, SIEVE cannot control the invocation of `onServiceDisconnected()`. As a result, there are 16 unreached races caused due to this reason.

- **Crashes.** There are 7 crashes listed in Column "#Crashes", causing 7 unreached races. This can happen, as described in Section 4.4.4, since SIEVE makes its decisions to instrument a conditional by reasoning about its related control and data dependences, without understanding the programmers' intended correlation

between a conditional and its guarded racy statement(s). How to reason about implicit information flow [37] remains to be a challenging problem. Note that we have not observed any infinite loops introduced by SIEVE.

- **Others.** The remaining 11 unreached races happen (as desired), due to the correct handling of these races in the code by having either avoided a race by using synchronization or handled a race by catching the run-time exception thrown.

In Column "#Reached Races" for RQ1, we give the number of reached races, together with the number of dangerous races reported by SIEVE (in parentheses), for each app. If SIEVE can successfully reach the two racy statements of a suspicious race in a correct order (with the free statement preceding the use statement), then the race is considered to be reached. For a reached race, if the app throws a null pointer exception, SIEVE will report a dangerous race.

In general, for the 190 suspicious event-based races reported statically in the 25 apps, SIEVE has successfully reached 90 (47.3%) of them and reported 16 dangerous races, which are all true races. Among the remaining 100 races, which consist of 19 infeasible races and 81 unreached races, we only miss 2 true races due to hidden UI Widgets as described earlier. This demonstrates that SIEVE with selective branch instrumentation can effectively expose event-based races.

4.5.3.2 RQ2. Improvement

We assess how SIEVE advances the state of the art [23] in detecting event-based races dynamically by comparing SIEVE with SIEVE-ZB (a version of SIEVE without performing any branch instrumentation).

Let us now analyze our results in Column "RQ2" to see how much more effective that SIEVE is than SIEVE-ZB. Due to the lack of branch analysis, SIEVE-ZB reports no infeasible races at all. By performing no branch instrumentation, SIEVE-ZB

VitoshaDM

```
1.  public void onCreate(){
2.   this.rollMode = false;
3.   this.sounds = new SoundPool();
4.   Timer timer = new Timer();
5.   timer.schedule(new TimerTask(){
6.    public void run(){
7.     roll();
8.    }}, 0, 100);}

9.  public void roll(){
10.  if(rollMode){
11.   this.sounds.play();
12.   this.rollMode = false;}}

13. public void onClick(){
14.  this.rollMode = true;}
15. public void onDestroy(){
16.  this.sounds = null;}
```

Figure 4.9: A true race missed in `VitoshaDM` by Sieve-ZB but exposed by Sieve.

reaches only 56 (29.4%) of the 190 suspicious races in the 25 apps. With selective branch instrumentation, SIEVE has reached 90 suspicious races, representing an increase of 60.7% and reports 2 more true races in `VitoshaDM`.

Figure 4.9 illustrates a true race in `VitoshaDM` missed by SIEVE-ZB but exposed by SIEVE. This race happens between lines 11 and 16. On executing `onCreate()`, a `TimerTask` will be scheduled to run in a background thread (lines 5-8), which invokes `roll()` every 100 milliseconds. During the iterative execution of `roll()`,

if `onDestroy()` is invoked, `this.sounds` will be freed at line 16, causing a null pointer exception to be thrown at line 11. SIEVE-ZB has missed this race, since it can only reach line 11 after `this.rollMode` has been set to true earlier by `onClick()`. However, this did not happen as it failed to generate an event to invoke `onClick()` that way. In contrast, SIEVE has exposed this race successfully. Due to our branch analysis, the conditional at line 10 will be instrumented (as always taken). During our dynamic execution, assisted by the semaphore mechanism discussed in Section 4.4.5, SIEVE has exposed this race as a true race (due to a null pointer exception observed).

Figure 4.10 also compares SIEVE and SIEVE-ZB in terms of a so-called reach rate for 16 apps (excluding the 9 apps without any reached races found by both tools). For each tool, its *reach rate* for an app is the percentage of the reached races found by the tool over the total number of suspicious races in the app. For the 16 apps compared, SIEVE-ZB always achieves no better reach rates, obtaining strictly lower reach rates than SIEVE in 8 apps with 0 reach rates in `Duerem`, `Ithaka Game`, `Risk App` and `VitoshaDM`. As a result, SIEVE has found 2 true races in `VitoshaDM` that are missed by SIEVE-ZB. On average, SIEVE's reach rate is 77% but SIEVE-ZB's is only 46% per app. This has allowed SIEVE to expose more true races than SIEVE-ZB successfully.

4.5.3.3 RQ3. Selective Instrumentation

We demonstrate the advantages of SIEVE by comparing it with SIEVE-FB, a modified version of SIEVE. Like SIEVE, SIEVE-FB also performs instrumentation on conditionals to steer to execution towards desired branches and racy statements. Unlike SIEVE, however, SIEVE-FB instruments all the guarding conditionals for a suspicious race (i.e., with full branch instrumentation). With selective branch

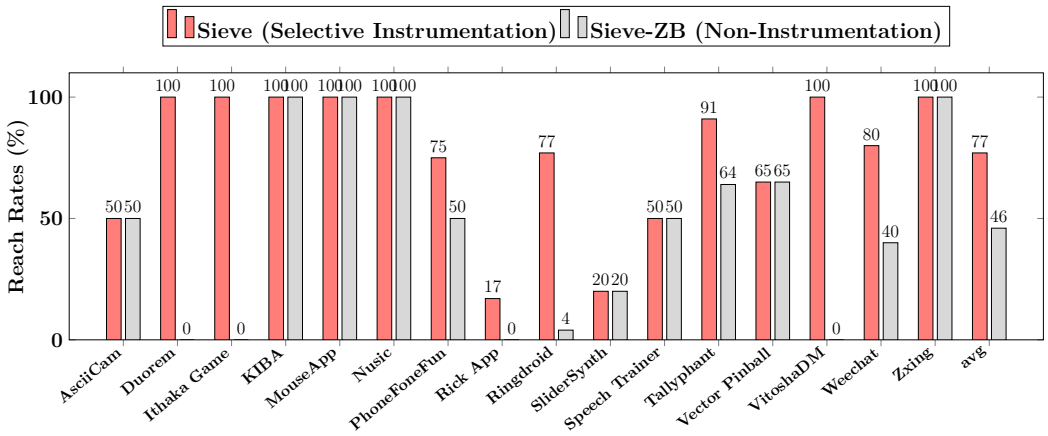


Figure 4.10: Comparing Sieve and Sieve-ZB in terms of their reach rates.

instrumentation, Sieve exposes the 16 true races with no false positives reported. With full branch instrumentation, Sieve-FB has only exposed 15 true races but issued 14 false alarms (with a false positive rate of 48.3%).

Let us now analyze our results given in Column "RQ3". Like Sieve-ZB, Sieve-FB also does not discover any infeasible races at all (due to the lack of branch analysis). There are two reasons why Sieve is more effective than Sieve-FB:

- **False Positives.** Sieve-FB is significantly less precise than Sieve in terms of the percentage of reached races established through infeasible paths. Among the 86 reached races found by Sieve-FB (4 fewer than that found by Sieve), the program paths leading to many reached races are infeasible. This is reflected by the fact that Sieve has reported 16 races with no false positives at all but Sieve-FB has reported 29 races including 14 false positives.
- **Program Crashes.** By instrumenting all guarding conditionals for a suspicious race indiscriminately without considering safeness, Sieve-FB has induced significantly more crashes than Sieve: 49 in 15 apps for Sieve-FB relative to 7 in 3 apps for Sieve. Note that Sieve-FB has reached fewer races than Sieve in `Ithaka Game`, `Ringdroid`, `SliderSynth`, `Zxing` and missed one true race in `KIBA`

due to more exceptions thrown. In `Speech Trainer`, SIEVE-FB reaches fewer races without any crash due to the infinite loop introduced by instrumentation.

Ringdroid

```
1.  public void onCreate(){
2.   this.mPlayer = null;
3.   this.mIsPlaying = false;
4.   load();}

5.  public void load(){
     ...
6.   update();
7.   mPlayer = new SamplePlayer();}

8.  public void update(){
9.   if(mIsPlaying) ──────► if(true)
10.   mPlayer.getCurrentPosition();}

11. public void onClick(){
     ...
12.  update();}
```

Figure 4.11: A false positive in `Ringdroid` reported by Sieve-FB but avoided by Sieve.

Figure 4.11 illustrates a false positive in `Ringdroid` reported by SIEVE-FB but avoided by SIEVE, on `mPlayer` between lines 2 and 10. During the normal execution starting from `onCreate()`, `mPlayer` is assigned with a new object at line 7 after being nullified at line 2. Therefore, invoking `onClick()` later will never cause a null pointer exception at line 10. However, with full branch instrumen-

tation, SIEVE-FB will consider the conditional at line 9 as being always taken. On executing `onCreate()`, lines 2 and 10 will be reached in that order, causing a null pointer exception at line 10. This is then reported as a race SIEVE-FB. As for SIEVE, when analyzing the conditional at line 9 reachable from `onClick()`, it finds that the conditional can also be reached from `onCreate()`, which happens before `onClick()`. According to [S-SAFENESS] in Figure 4.6, SIEVE will not instrument this conditional (since it is unsafe), avoiding the false positive reported by SIEVE-FB.

4.5.4 Limitations

SIEVE can be improved along a number of directions. First, SIEVE's branch analysis leverages the call graph built by FLOWDROID [14] and the SPARK [38] pointer analysis framework provided in Soot [71]. Currently, FLOWDROID does not support inter-component communications (ICC) analysis and handles Java reflection partially using Soot. Therefore, the call graph it builds may be incomplete. To resolve this problem, ICC analyses [17, 39, 57, 58] and reflection analyses [15, 27, 40, 43, 45, 81] are needed. As for the SPARK pointer analysis, it only performs a flow- and context-insensitive may-alias analysis conservatively, affecting the precision of our branch analysis. This can be improved by incorporating more powerful pointer analyses [36, 41, 42, 46, 53, 65, 67–69].

Second, SIEVE is conservative in estimating the values of certain variables (e.g., those obtained from operations on primitive types as in [V-RESERVE] in Figure 4.5). This can be improved by leveraging an SMT solver such as Z3 [20]. Although SIEVE has achieved a high recall rate of 88.9% for the 25 apps evaluated, a more precise value range analysis will enable us to instrument more conditionals, and consequently, reach and expose more true races in more apps.

Finally, during the dynamic execution phase, SIEVE cannot yet trigger all callback methods or system methods as discussed in Section 4.5.3.1. This can be enhanced with more engineering efforts.

4.6 Chapter Summary

We have introduced a new approach, SIEVE, for exposing event-based races dynamically in Android apps based on a new selective branch instrumentation technique. In comparison with the state of the art, SIEVE can expose the event-based races in real-world Android apps more effectively, making it easily deployable as a practical bug-detection tool.

Chapter 5

Conclusions and Future Work

As concurrency bugs in mobile applications are pernicious and prevalent, there is an urgent need for effective concurrency bug detection tools. The hybrid concurrency system, comprising both traditional multi-threading and event dispatching in mobile platforms, makes concurrency bug detection especially challenging. Dynamic analysis techniques suffer from the code coverage problem because of their dynamic nature, although they can achieve better precision. Static analysis techniques can be more sound, but are prone to high false positive rates. To tackle these challenges, we have presented two practical tools in this thesis, Sard and Sieve, which outperform prior state-of-the-art tools in detecting event-based concurrency bugs.

In Chapter 3, we proposed Sard, a novel static analysis tool for detecting use-after-free order violations in Android apps. Targeting the hybrid concurrency system of Android that introduces both intra- and inter-thread order violations, Sard systematically models the asynchronous events and applies a flow- and context-sensitive happens-before analysis to reason about the happens-before relations between interleaved events. With the help of precise happens-before relations, Sard

is able to report less false positives and detect more true UAF order violations due to removing the falsely established happens-before relations in previous tools.

In Chapter 4, we proposed Sieve, a tool for exposing event-based races dynamically in Android apps. Sieve first applies the static analysis tool Sard to find all suspicious races, and then exposes them dynamically to reduce false positives. A novel selective branch instrumentation technique is leveraged to overcome the challenge that conditional statements in an app may prevent the dynamic execution from reaching the target statements. Sieve selectively instruments some conditionals by fixing the true/false outcomes of them based on a systematic branch analysis so that it can expose event-based races effectively while reducing the negative impacts of instrumentation.

We implemented Sard and Sieve and evaluated them using real-world Android applications. The experimental results validated their effectiveness and efficiency in detecting event-based races. However, both Sard and Sieve can be extended in several directions.

Although leveraging flow- and context-sensitive happens-before analysis, Sard still reports many false positives. To improve this, Sard could identity more implicit happens-before relations through a more systematic analysis on some framework APIs. In addition, our lightweight path-sensitive analysis on finding UAF-infeasible paths could be improved with better data-flow analysis.

As for the limitations of Sieve, an SMT solver could be used to improve the value range analysis so that Sieve can select more conditionals for instrumentation to expose more true races in apps. Second, Sieve cannot yet execute all applications ideally during its dynamic execution phase. Therefore, more engineering effort can be made to improve Sieve 's performance.

Finally, SARD and SIEVE share some common features that can be improved together. First, both of them work with a pointer analysis. Therefore, better pointer analysis tools that are equipped with context-sensitivity, type-sensitivity and reflection analysis will enhance their effectiveness. Second, SARD and SIEVE are designed for Android apps. Thus, an ICC (Inter-Component Communication) analysis can allow both to analyze some parts of an app that would otherwise be missed.

Bibliography

[1] *Android Debug Bridge: a versatile command-line tool*. https://developer.android.com/studio/command-line/adb.

[2] *The Android software stack*. https://developer.android.com/guide/platform.

[3] *ConnectBot: SSH and local shell client*. https://f-droid.org/packages/org.connectbot/.

[4] *CVE-2017-0780*. https://www.cvedetails.com/cve/CVE-2017-0780/.

[5] Digital 2019: Global internet use accelerates. https://wearesocial.com/blog/2019/01/digital-2019-global-internet-use-accelerates.

[6] *F-Droid: Free and Open Source Android App Repositoryf*. https://f-droid.org/.

[7] *Google Play App Store*. https://play.google.com/store.

[8] *Google, UI/Application Exerciser Monkey*. https://developer.android.com/studio/test/monkey.

[9] *Logcat*. https://developer.android.com/studio/command-line/logcat.

[10] Mobile & Tablet Operating System Market Share Worldwide. https://gs.statcounter.com/os-market-share/mobile-tablet/worldwide/.

[11] *Number of available apps in Google Play Store.* https://www.statista.com/statistics/266210/number-of-available-applications-in-the-google-play-store/.

[12] *PhoneFoneFun: a playphone for toddlers.* https://f-droid.org/packages/com.quaap.phonefonefun/.

[13] *VitoshaDM: helps you to make decisions.* https://f-droid.org/packages/eu.veldsoft.vitoshadm/.

[14] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, page 259–269, 2014.

[15] P. Barros, R. Just, S. Millstein, P. Vines, W. Dietl, M. d'Amorim, and M. D. Ernst. Static analysis of implicit control flow: Resolving java reflection and android intents (t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 669–679, 2015.

[16] P. Bielik, V. Raychev, and M. Vechev. Scalable race detection for android applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2015, page 332–348, 2015.

[17] A. Bosu, F. Liu, D. D. Yao, and G. Wang. Collusive data leak and more: Large-scale threat analysis of inter-app communications. In *Proceedings of the*

2017 ACM on Asia Conference on Computer and Communications Security, ASIA CCS '17, page 71–85, 2017.

[18] C. Boyapati, R. Lee, and M. Rinard. Ownership Types for Safe Programming: Preventing Data Races and Deadlocks. In *Proceedings of the 17th ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, 2002.

[19] W. Choi, G. Necula, and K. Sen. Guided gui testing of android apps with minimal restart and approximate learning. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA '13, page 623–640, 2013.

[20] L. de Moura and N. Bjørner. Z3: An efficient smt solver. In C. R. Ramakrishnan and J. Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer Berlin Heidelberg, 2008.

[21] D. Engler and K. Ashcraft. RacerX: Effective, Static Detection of Race Conditions and Deadlocks. *SIGOPS Oper. Syst. Rev.*, pages 237–252, 2003.

[22] J. Erickson, M. Musuvathi, S. Burckhardt, and K. Olynyk. Effective Data-race Detection for the Kernel. In *Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation*, 2010.

[23] L. Fan, T. Su, S. Chen, G. Meng, Y. Liu, L. Xu, and G. Pu. Efficiently manifesting asynchronous programming errors in android apps. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, ASE 2018, page 486–497, 2018.

[24] C. Flanagan and S. N. Freund. Type-based Race Detection for Java. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*, 2000.

[25] C. Flanagan and S. N. Freund. FastTrack: Efficient and Precise Dynamic Race Detection. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2009.

[26] X. Fu, D. Lee, and C. Jung. nAdroid: Statically detecting ordering violations in android applications. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, CGO 2018, page 62–74, 2018.

[27] N. Grech, G. Kastrinis, and Y. Smaragdakis. Efficient Reflection String Analysis via Graph Coloring. In *32nd European Conference on Object-Oriented Programming (ECOOP 2018)*, Leibniz International Proceedings in Informatics (LIPIcs), pages 26:1–26:25.

[28] S. Hao, B. Liu, S. Nath, W. G. Halfond, and R. Govindan. Puma: Programmable ui-automation for large-scale dynamic analysis of mobile apps. In *Proceedings of the 12th Annual International Conference on Mobile Systems, Applications, and Services*, MobiSys '14, page 204–217, 2014.

[29] T. A. Henzinger, R. Jhala, and R. Majumdar. Race checking by context inference. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation*, PLDI '04, page 1–13, 2004.

[30] C.-H. Hsiao, S. Narayanasamy, E. M. I. Khan, C. L. Pereira, and G. A. Pokam. Asyncclock: Scalable inference of asynchronous event causality. In *Proceedings of the Twenty-Second International Conference on Architectural Support for*

Programming Languages and Operating Systems, ASPLOS '17, page 193–205, 2017.

[31] C.-H. Hsiao, J. Yu, S. Narayanasamy, Z. Kong, C. L. Pereira, G. A. Pokam, P. M. Chen, and J. Flinn. Race detection for event-driven mobile applications. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, page 326–336, 2014.

[32] Y. Hu and I. Neamtiu. Static detection of event-based races in android apps. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '18, page 257–270, 2018.

[33] Y. Hu, I. Neamtiu, and A. Alavi. Automatically verifying and reproducing event-based races in android apps. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ISSTA 2016, page 377–388, 2016.

[34] J. Huang, P. O. Meredith, and G. Rosu. Maximal sound predictive race detection with control flow abstraction. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, page 337–348, 2014.

[35] J. Huang and A. K. Rajagopalan. Precise and maximal race detection from incomplete traces. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2016, page 462–476, 2016.

[36] S. Jeong, M. Jeon, S. Cha, and H. Oh. Data-driven context-sensitivity for points-to analysis. *Proc. ACM Program. Lang.*, (OOPSLA), 2017.

[37] D. King, B. Hicks, M. Hicks, and T. Jaeger. Implicit flows: Can't live with 'em, can't live without 'em. In *International Conference on Information Systems Security*, pages 56–70, 2008.

[38] O. Lhoták and L. Hendren. Scaling java points-to analysis using spark. In *Proceedings of the 12th International Conference on Compiler Construction*, CC'03, page 153–169, 2003.

[39] L. Li, A. Bartel, T. F. Bissyandé, J. Klein, Y. Le Traon, S. Arzt, S. Rasthofer, E. Bodden, D. Octeau, and P. McDaniel. Iccta: Detecting inter-component privacy leaks in android apps. In *Proceedings of the 37th International Conference on Software Engineering - Volume 1*, ICSE '15, page 280–291, 2015.

[40] L. Li, T. F. Bissyandé, D. Octeau, and J. Klein. Droidra: Taming reflection to support whole-program analysis of android apps. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ISSTA 2016, page 318–329, 2016.

[41] Y. Li, T. Tan, A. Møller, and Y. Smaragdakis. Precision-guided context sensitivity for pointer analysis. *Proc. ACM Program. Lang.*, (OOPSLA), 2018.

[42] Y. Li, T. Tan, A. Møller, and Y. Smaragdakis. Scalability-first pointer analysis with self-tuning context-sensitivity. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2018, page 129–140, 2018.

[43] Y. Li, T. Tan, and J. Xue. Effective soundness-guided reflection analysis. In S. Blazy and T. Jensen, editors, *Static Analysis*, pages 162–180, 2015.

[44] Y. Lin, C. Radoi, and D. Dig. Retrofitting concurrency for android applications through refactoring. In *Proceedings of the 22nd ACM SIGSOFT International*

Symposium on Foundations of Software Engineering, FSE 2014, page 341–352, 2014.

[45] J. Liu, Y. Li, T. Tan, and J. Xue. Reflection analysis for java: Uncovering more reflective targets precisely. In *2017 IEEE 28th International Symposium on Software Reliability Engineering (ISSRE)*, pages 12–23, 2017.

[46] J. Lu and J. Xue. Precision-preserving yet fast object-sensitive pointer analysis with partial context sensitivity. *Proc. ACM Program. Lang.*, (OOPSLA), Oct. 2019.

[47] B. Lucia and L. Ceze. Cooperative Empirical Failure Avoidance for Multi-threaded Programs. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2013.

[48] A. Machiry, R. Tahiliani, and M. Naik. Dynodroid: An input generation system for android apps. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2013, page 224–234, 2013.

[49] R. Mahmood, N. Mirzaei, and S. Malek. Evodroid: Segmented evolutionary testing of android apps. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2014, page 599–609, 2014.

[50] P. Maiya and A. Kanade. Efficient computation of happens-before relation for event-driven programs. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2017, page 102–112, 2017.

[51] P. Maiya, A. Kanade, and R. Majumdar. Race detection for android applications. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, page 316–325, 2014.

[52] K. Mao, M. Harman, and Y. Jia. Sapienz: Multi-objective automated testing for android applications. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ISSTA 2016, page 94–105, 2016.

[53] A. Milanova, A. Rountev, and B. G. Ryder. Parameterized object sensitivity for points-to analysis for java. *ACM Trans. Softw. Eng. Methodol.*, page 1–41, 2005.

[54] N. Mirzaei, H. Bagheri, R. Mahmood, and S. Malek. Sig-droid: Automated system input generation for android applications. In *2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE)*, pages 461–471, 2015.

[55] N. Mirzaei, J. Garcia, H. Bagheri, A. Sadeghi, and S. Malek. Reducing combinatorics in gui testing of android applications. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*, pages 559–570, 2016.

[56] M. Naik, A. Aiken, and J. Whaley. Effective Static Race Detection for Java. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI'06, pages 308–319, 2006.

[57] D. Octeau, S. Jha, M. Dering, P. McDaniel, A. Bartel, L. Li, J. Klein, and Y. Le Traon. Combining static analysis with probabilistic models to enable market-scale android inter-component analysis. In *Proceedings of the 43rd*

Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '16, page 469–484, 2016.

[58] D. Octeau, D. Luchaup, M. Dering, S. Jha, and P. McDaniel. Composite constant propagation: Application to android inter-component communication analysis. In *Proceedings of the 37th International Conference on Software Engineering - Volume 1*, ICSE '15, page 77–88.

[59] P. Pratikakis, J. S. Foster, and M. Hicks. LOCKSMITH: Context-sensitive Correlation Analysis for Race Detection. In *Proceedings of the 27th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2006.

[60] S. Qadeer and D. Wu. KISS: Keep It Simple and Sequential. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation*, 2004.

[61] A. Sadeghi, R. Jabbarvand, and S. Malek. Patdroid: Permission-aware gui testing of android. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2017, page 220–232, 2017.

[62] G. Safi, A. Shahbazian, W. G. J. Halfond, and N. Medvidovic. Detecting event anomalies in event-based systems. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2015, page 25–37, 2015.

[63] S. Savage, M. Burrows, G. Nelson, P. Sobalvarro, and T. Anderson. Eraser: A Dynamic Data Race Detector for Multithreaded Programs. *ACM Trans. Comput. Syst.*, 15(4):391–411, 1997.

[64] T. Sheng, N. Vachharajani, S. Eranian, R. Hundt, W. Chen, and W. Zheng. RACEZ: A Lightweight and Non-invasive Race Detection Tool for Production Applications. In *Proceedings of the 33rd International Conference on Software Engineering*, 2011.

[65] Y. Smaragdakis, M. Bravenboer, and O. Lhoták. Pick your contexts well: Understanding object-sensitivity. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, page 17–30, 2011.

[66] T. Su, G. Meng, Y. Chen, K. Wu, W. Yang, Y. Yao, G. Pu, Y. Liu, and Z. Su. Guided, stochastic model-based gui testing of android apps. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ESEC/FSE 2017, page 245–256, 2017.

[67] Y. Sui and J. Xue. Svf: Interprocedural static value-flow analysis in llvm. In *Proceedings of the 25th International Conference on Compiler Construction*, CC 2016, page 265–266, 2016.

[68] T. Tan, Y. Li, and J. Xue. Making k-object-sensitive pointer analysis more precise with still k-limiting. In X. Rival, editor, *Static Analysis*, pages 489–510, 2016.

[69] T. Tan, Y. Li, and J. Xue. Efficient and precise points-to analysis: Modeling the heap by merging equivalent automata. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2017, page 278–291, 2017.

[70] H. Tang, G. Wu, J. Wei, and H. Zhong. Generating test cases to expose concurrency bugs in android applications. In *Proceedings of the 31st IEEE/ACM*

International Conference on Automated Software Engineering, ASE 2016, page 648–653, 2016.

[71] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan. Soot - a java bytecode optimization framework. In *Proceedings of the 1999 Conference of the Centre for Advanced Studies on Collaborative Research*, CASCON '99, page 13, 1999.

[72] K. Veeraraghavan, P. M. Chen, J. Flinn, and S. Narayanasamy. Detecting and Surviving Data Races Using Complementary Schedules. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, 2011.

[73] J. W. Voung, R. Jhala, and S. Lerner. RELAY: Static Race Detection on Millions of Lines of Code. In *Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering*, 2007.

[74] D. Wu, J. Liu, Y. Sui, S. Chen, and J. Xue. Precise static happens-before analysis for detecting uaf order violations in android. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, pages 276–287, 2019.

[75] J. Wu, H. Cui, and J. Yang. Bypassing races in live applications with execution filters. In *OSDI*, volume 10, pages 1–13, 2010.

[76] X. Xie and J. Xue. Acculock: Accurate and Efficient Detection of Data Races. In *Proceedings of the 9th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, 2011.

[77] S. Yang, D. Yan, H. Wu, Y. Wang, and A. Rountev. Static control-flow analysis of user-driven callbacks in android applications. In *2015 IEEE/ACM*

37th IEEE International Conference on Software Engineering, ICSE '15, pages 89–99, 2015.

[78] Y. Yu, T. Rodeheffer, and W. Chen. RaceTrack: Efficient Detection of Data Race Conditions via Adaptive Tracking. In *Proceedings of the Twentieth ACM Symposium on Operating Systems Principles*, 2005.

[79] T. Zhang, C. Jung, and D. Lee. ProRace: Practical Data Race Detection for Production Use. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017.

[80] T. Zhang, D. Lee, and C. Jung. TxRace: Efficient Data Race Detection Using Commodity Hardware Transactional Memory. *SIGOPS Oper. Syst. Rev.*, 50:159–173, 2016.

[81] Y. Zhang, T. Tan, Y. Li, and J. Xue. Ripple: Reflection analysis for android apps in incomplete information environments. In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*, CODASPY '17, page 281–288, 2017.

[82] B. Zhou, I. Neamtiu, and R. Gupta. A cross-platform analysis of bugs and bug-fixing in open source projects: Desktop vs. android vs. ios. In *Proceedings of the 19th International Conference on Evaluation and Assessment in Software Engineering*, EASE '15, 2015.

[83] B. Zhou, I. Neamtiu, and R. Gupta. Experience report: How do bug characteristics differ across severity classes: A multi-platform study. In *Proceedings of the 2015 IEEE 26th International Symposium on Software Reliability Engineering*, ISSRE '15, page 507–517, 2015.

[84] Q. Zhou, L. Li, L. Wang, J. Xue, and X. Feng. May-happen-in-parallel analysis with static vector clocks. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, CGO 2018, pages 228–240, 2018.